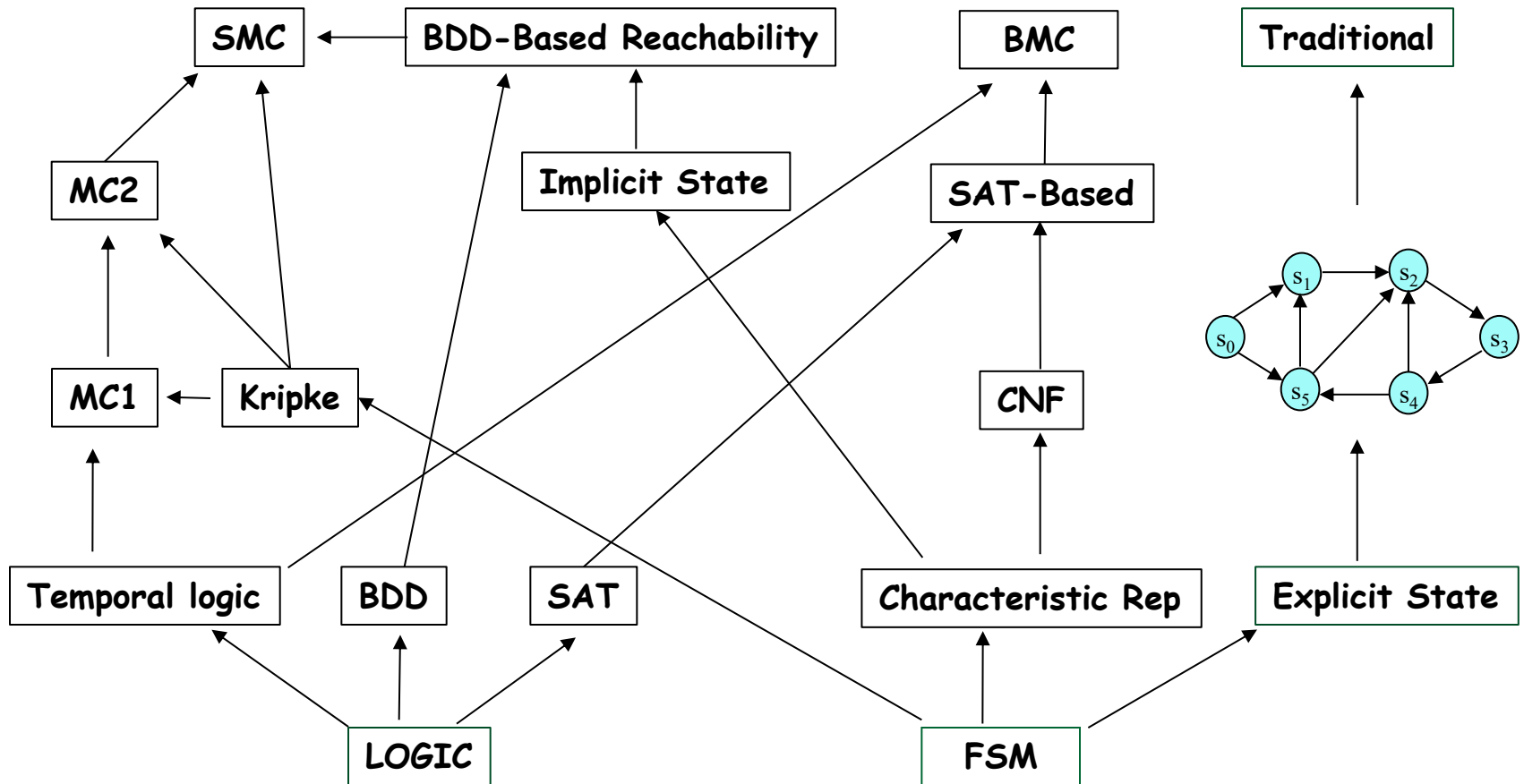




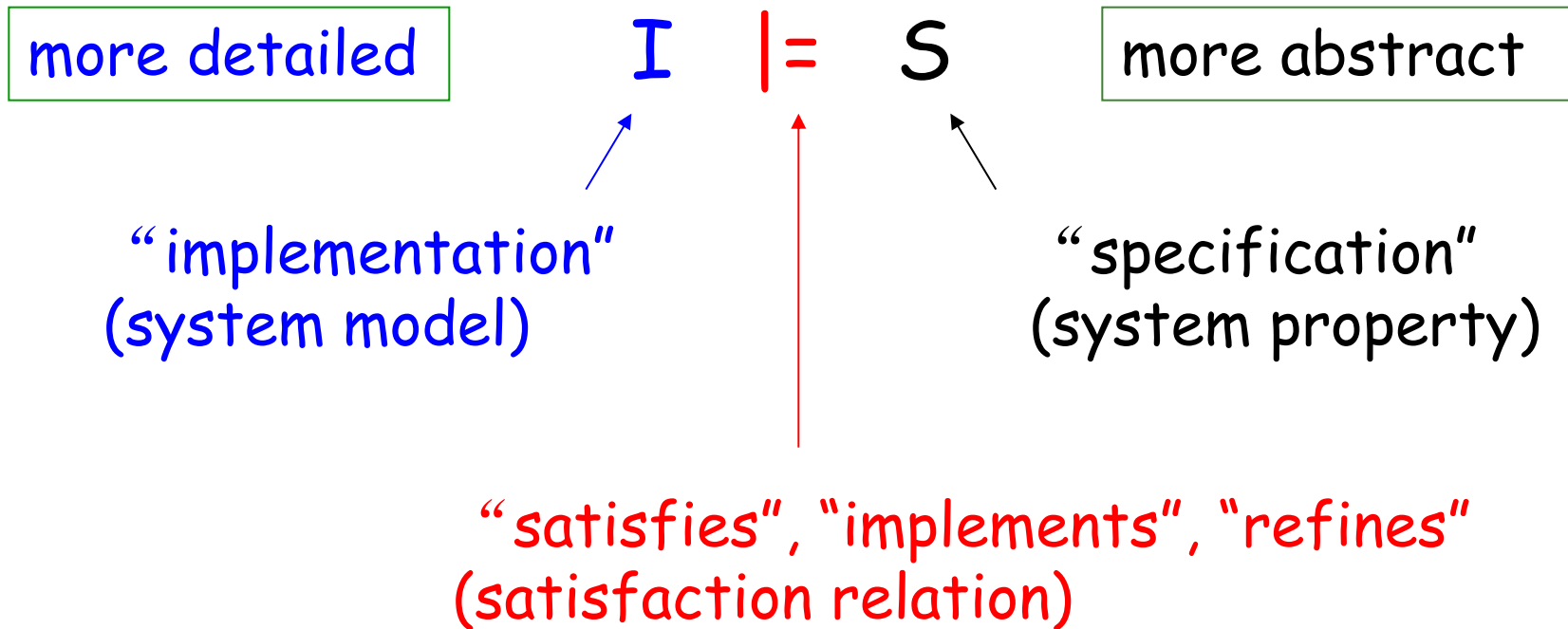
CTL Model Checking Problem



A Road Map



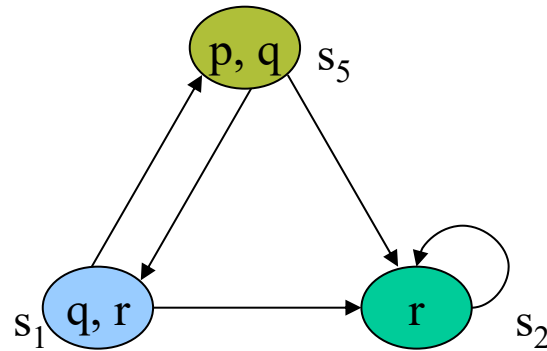
Problem Definition



Problem Definition

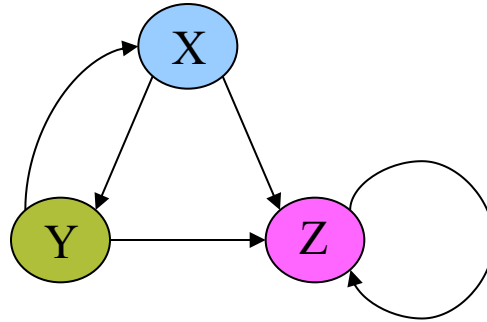
- Input:
 - A **Finite** Structure $M = (S, R, L)$
 - A temporal formula f
- What is a finite M ?

What is a finite M ?



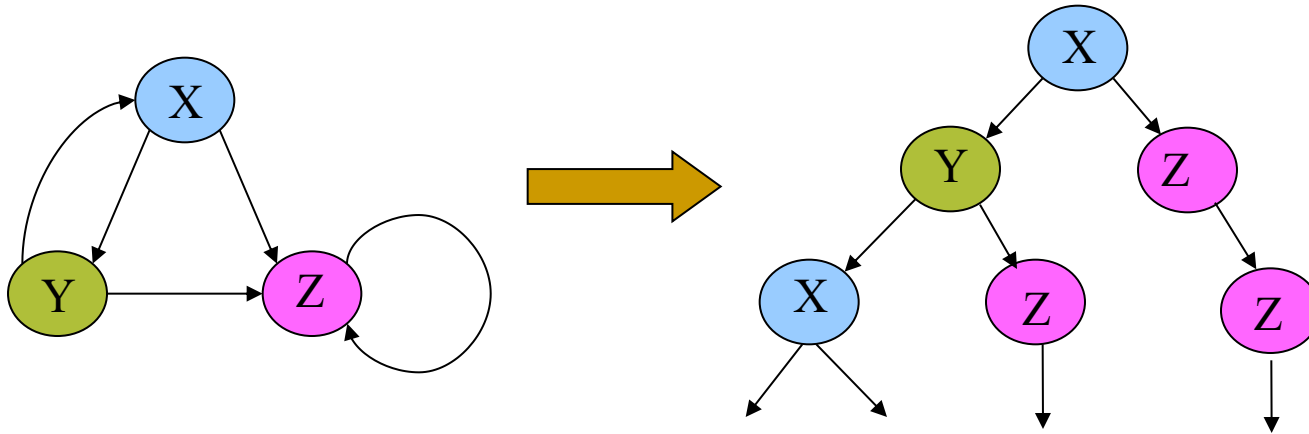
- A Kripke structure: $M = \langle S, R, L \rangle$
 - S : set of states
 - $R \subseteq S \times S$: transition relation
 - $L: S \rightarrow 2^{AP} : (\text{Truth valuation})$
 - Set of atomic propositions true in each state
 - There is at least one exit for each node.

What is a finite M ?



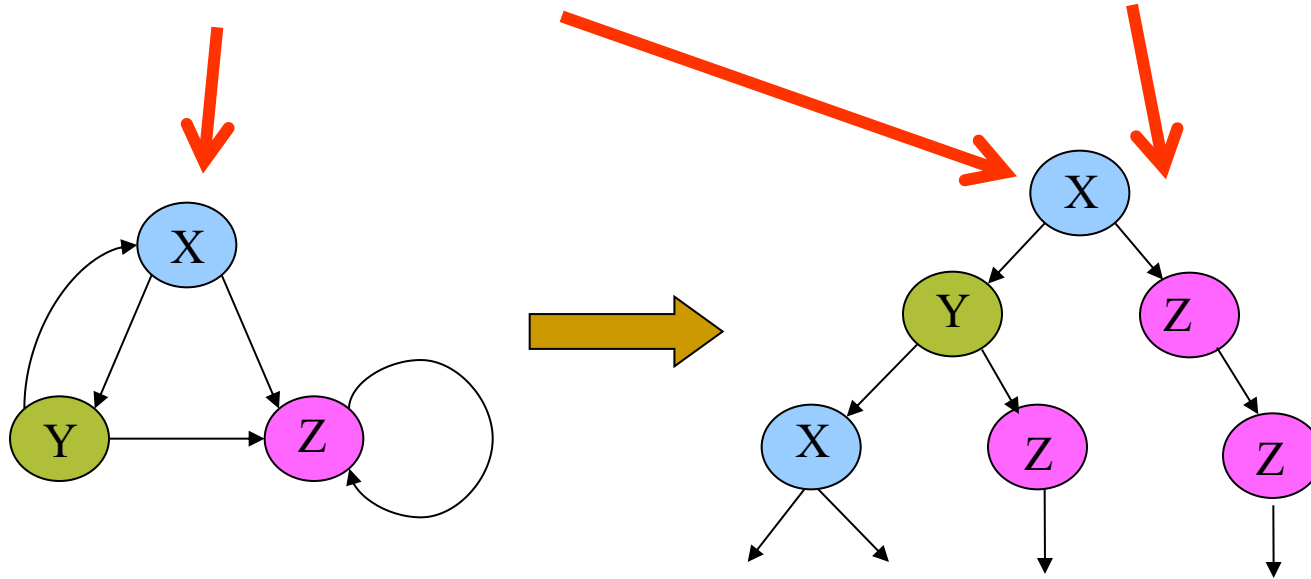
- A Finite Kripke Structure
- Why is this of interest to us?
 - The dynamics of a discrete system can be captured by a Kripke structure.

Computation Tree: An Infinite Tree

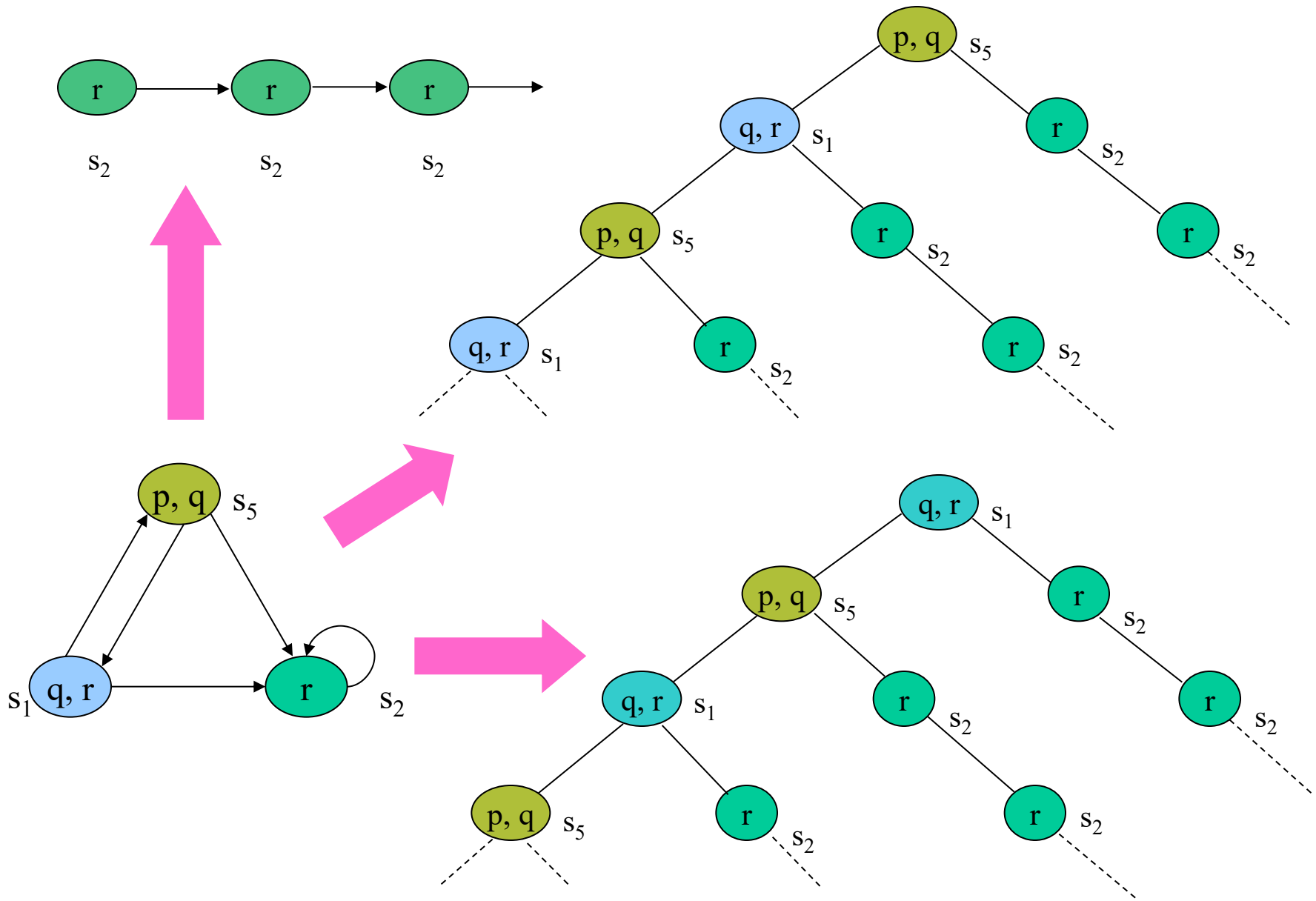


- If a state is designated as the initial state,
 - the structure can be unfolded into an infinite tree with that state as the root.
- Paths in the tree = possible computations or behaviors of the system

$M + \text{An Initial State} = B$

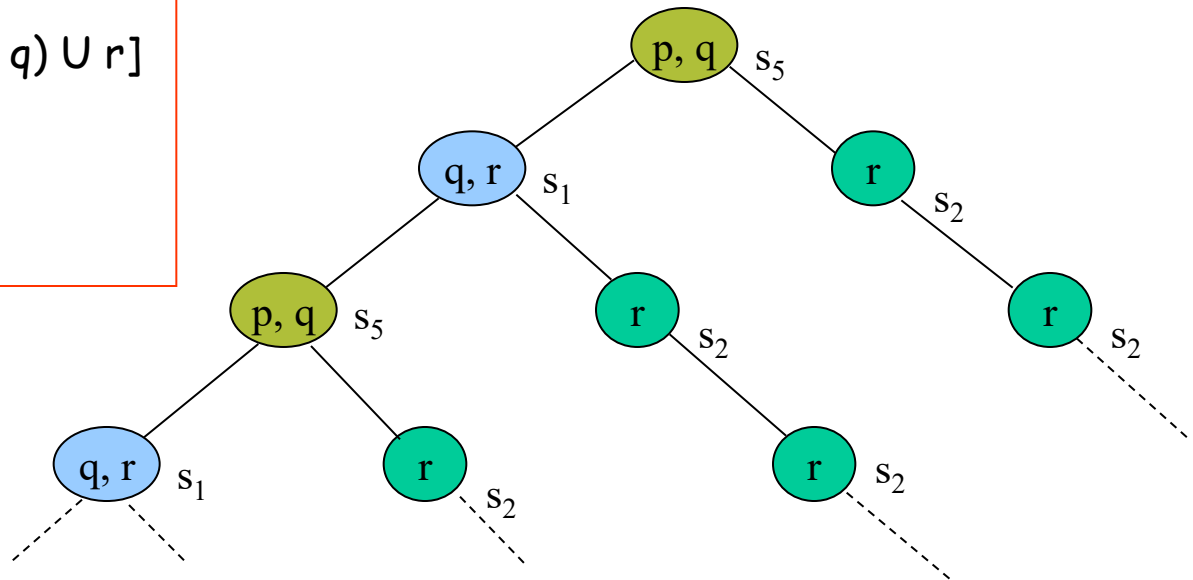
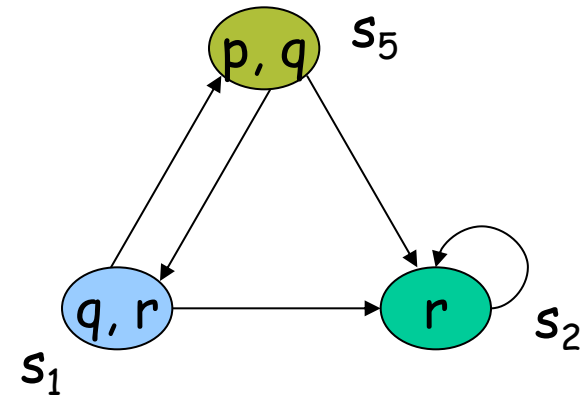


- A finite structure and a fixed initial state can determine an infinite tree.
- How many possible trees?
- Can we think of such a tree as an interpretation in CTL?

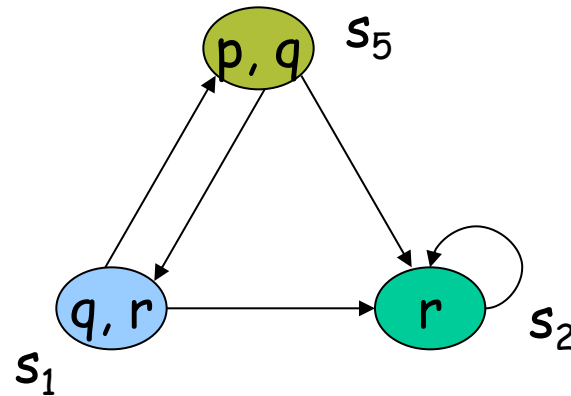


Interpretations of Formulas in M

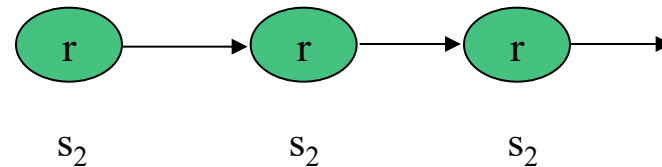
- $M, s_5 \models p \wedge q$
- $M, s_5 \models \neg r$
- $M, s_5 \models \text{True}$
- $M, s_5 \models \text{EX } (q \wedge r)$
- $M, s_5 \not\models \text{AX } (q \wedge r)$
- $M, s_5 \models \text{AF } r$
- $M, s_5 \models \text{EFE } [(p \wedge q) \cup r]$
- $M, s_5 \models \text{A } [p \cup r]$
- $M, s_5 \not\models \text{EF } (p \wedge r)$



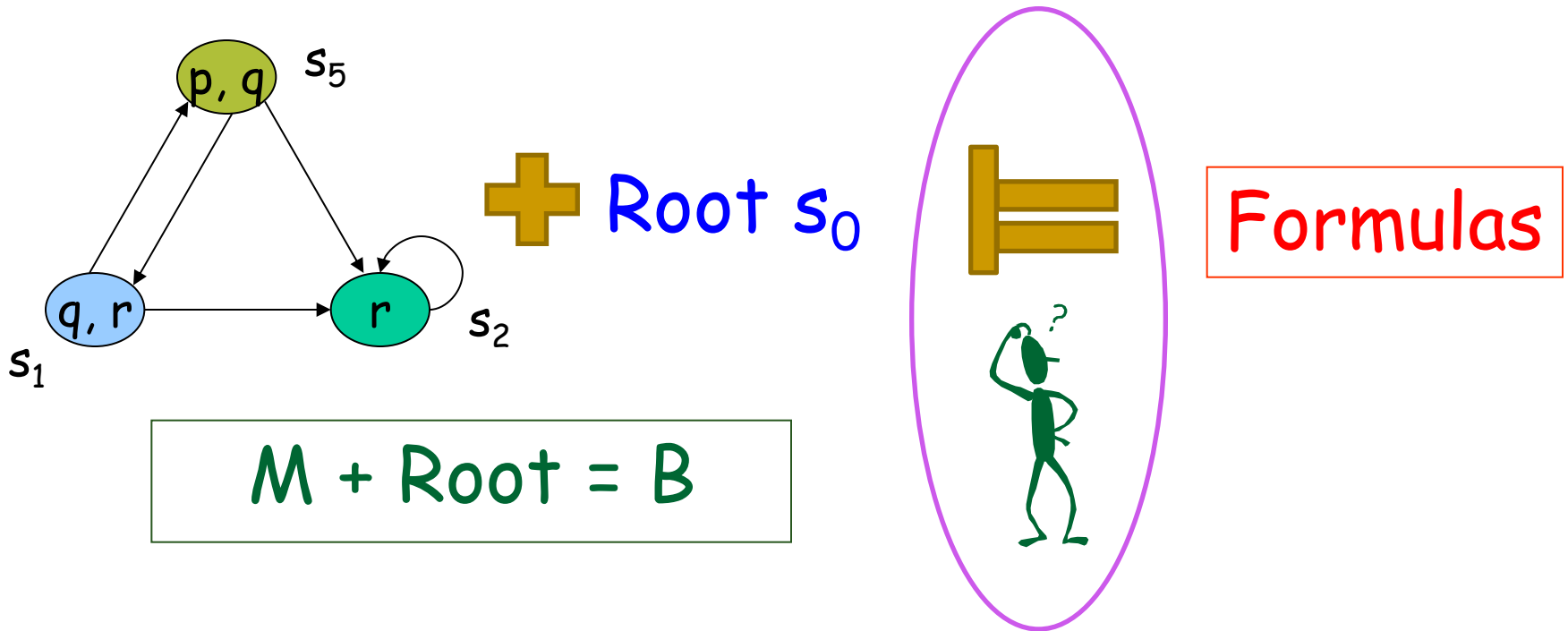
Interpretation of Formulas in M



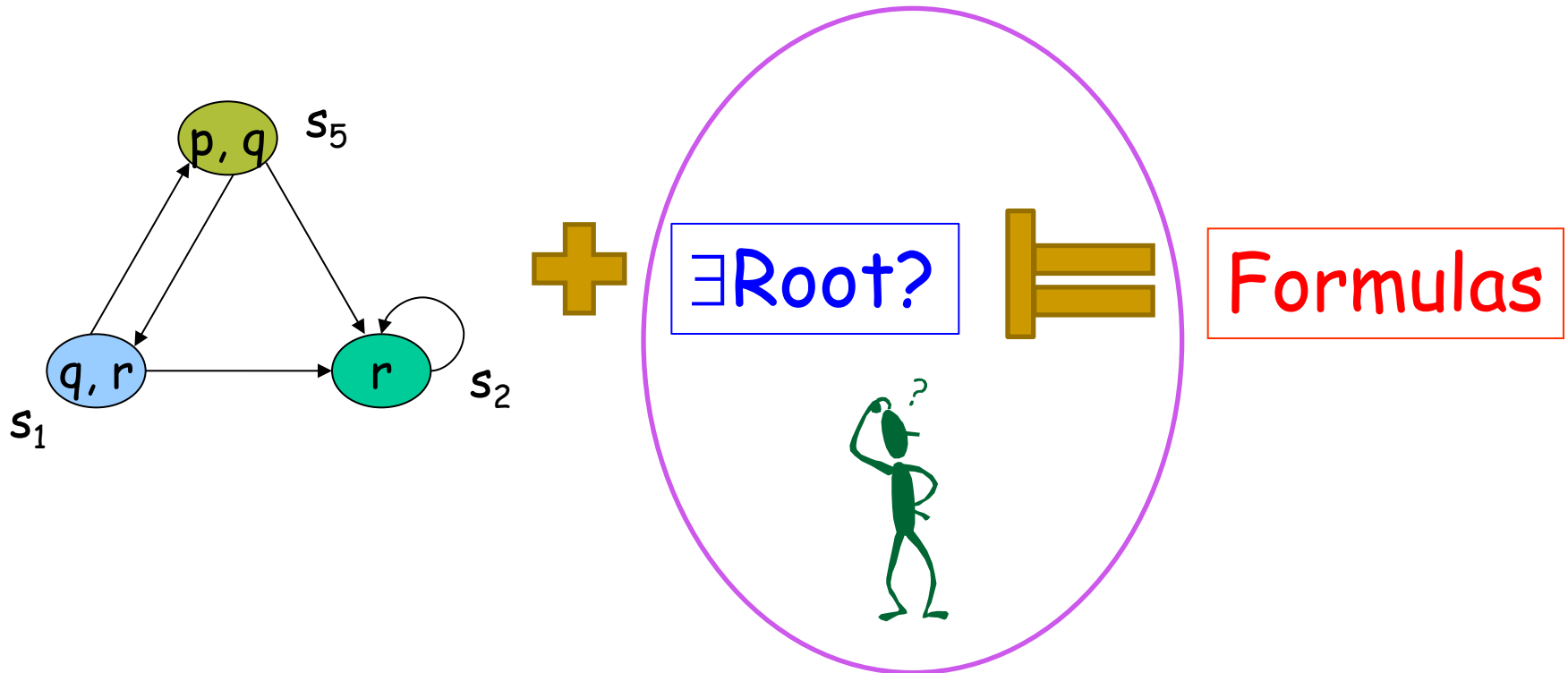
- $M, s_2 \models EG\ r$
- $M, s_2 \models AG\ r$
- $M, s_2 \not\models EF\ p$
- $M, s_2 \models AX\ r$
- $M, s_2 \not\models E(rUq)$



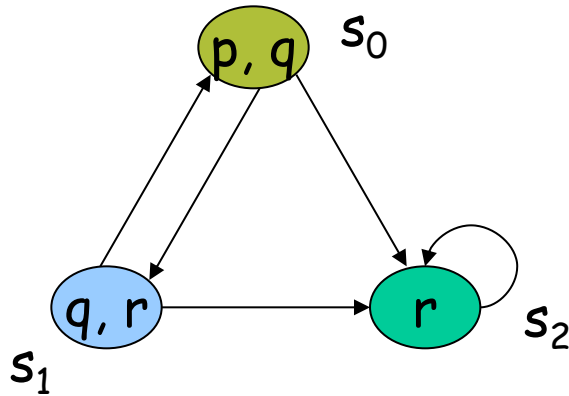
Interpretations of CTL Formulas in M



Model Checking of CTL Formulas in M

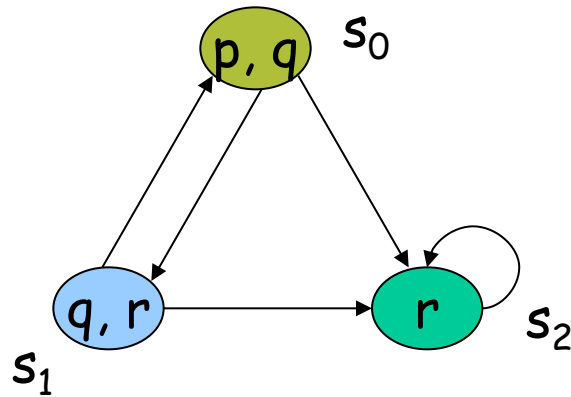


Model Checking in M



- $M, s_0 \models p \wedge q$
- $M, s_0 \models \neg r$
- $M, s_0 \models \text{True}$
- $M, s_0 \models \text{EX } (q \wedge r)$
- $M, s_0 \models \neg \text{AX } (q \wedge r)$
- $M, s_0 \models \neg \text{EF } (p \wedge r)$
- $M, s_2 \models \text{EG } r$
- $M, s_2 \models \text{AG } r$
- $M, s_0 \models \text{AF } r$
- $M, s_0 \models \text{EFE } [(p \wedge q) \text{ U } r]$
- $M, s_0 \models \text{A } [p \text{ U } r]$

Model Checking in M



???

- $M, s_0 \models p \wedge q$
- $M, s_0 \models \neg r$
- $M, s_0 \models \text{True}$
- $M, s_0 \models \text{EX } (q \wedge r)$
- $M, s_0 \models \neg \text{AX } (q \wedge r)$
- $M, s_0 \models \neg \text{EF } (p \wedge r)$
- $M, s_2 \models \text{EG } r$
- $M, s_2 \models \text{AG } r$
- $M, s_0 \models \text{AF } r$
- $M, s_0 \models \text{EFE } [(p \wedge q) \text{ U } r]$
- $M, s_0 \models \text{A } [p \text{ U } r]$

Problem Definition

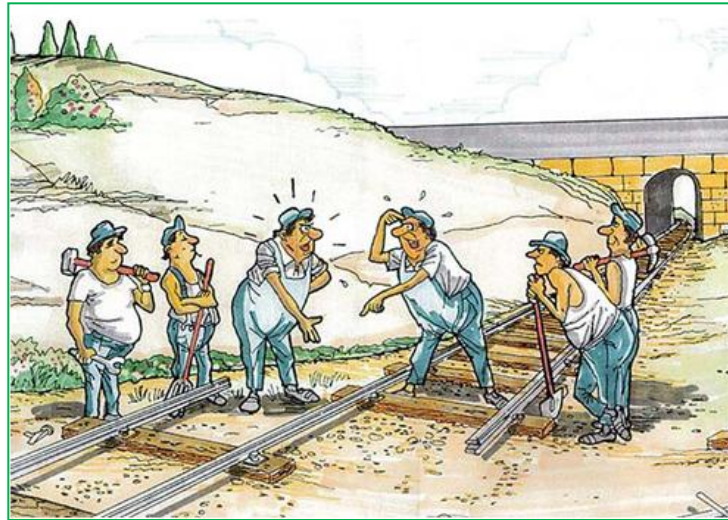
- Input:
 - A Finite Structure $M = (S, R, L)$
 - a CTL formula f .
- Output: The set of states of M which satisfy f ,
- Task: find all s_0 such that $M, s_0 \models f$

Model Checking Problem

- Given
 - a **finite** structure M and
 - a temporal logic formula p ,
 - Does M define a model of p ?
-
- Determine the truth of a formula with respect to a given state in a given “finite” model.
 - Find all state s of M such that $M, s \models f$.

Problem Complexity

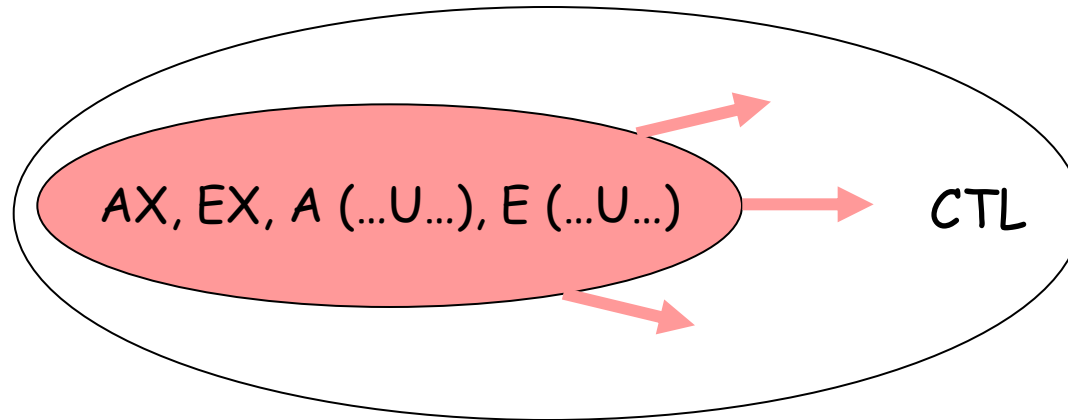
- For any propositional temporal logic, the model checking problem is *decidable*:
 - exhaustive search of all paths through the **finite** input structure



Problem Complexity

- The model checking for CTL is in deterministic polynomial time. [Wolper, 86].
- The model checking problem for PLTL is PSPACE-complete [Sistla, Clark, 85].
- The model checking problem for CTL* is PSPACE-complete [Clark, Emerson, Sistla, 86].
- Given any model-checking algorithm for a linear logic LTL, there is a model checking algorithm for the corresponding branching logic BTL, whose basic modalities are defined by the LTL, of the same order of complexity [Emerson, Lei, 87].

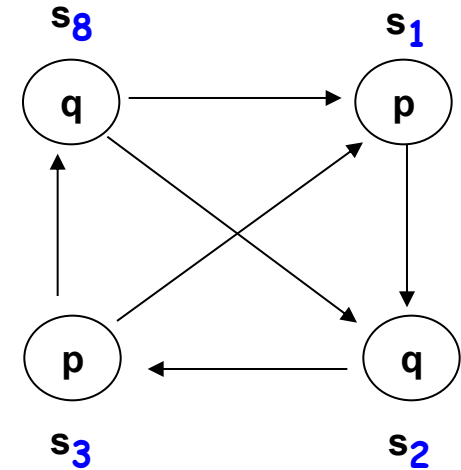
Devise Algorithms for A CTL Generator



$AFg = A(\text{true} \cup g)$
 $EFg = E(\text{true} \cup g)$
 $AGf = \neg E(\text{true} \cup \neg f)$
 $EGf = \neg A(\text{true} \cup \neg f)$

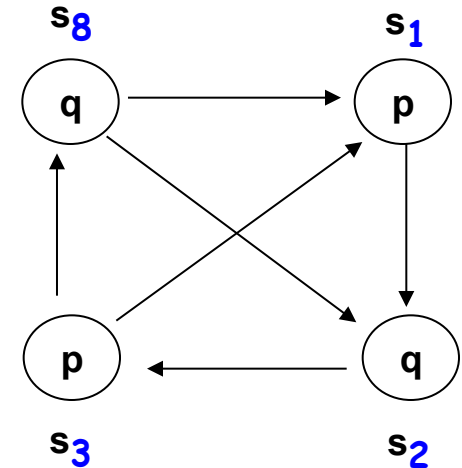
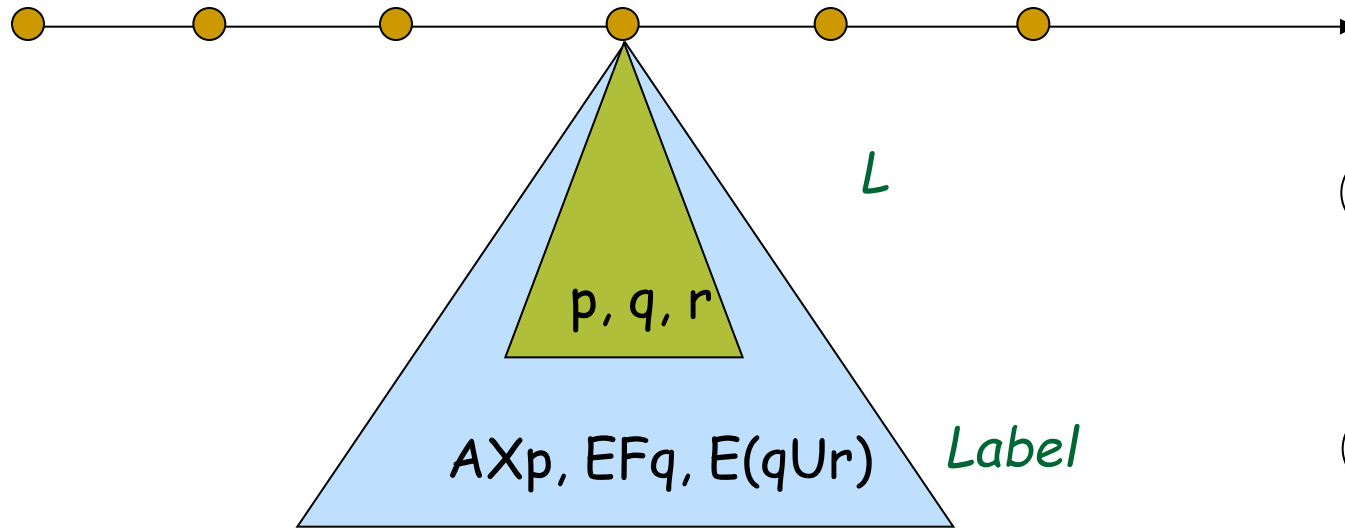
CTL Formulas from the Generator

$f =$ **AF**, EU, EX, $\wedge$, $\neg$ and False



- $\{s_1, s_3\} \models p$
- $\{???\} \models \text{AFEX } p$

Extend L to Label

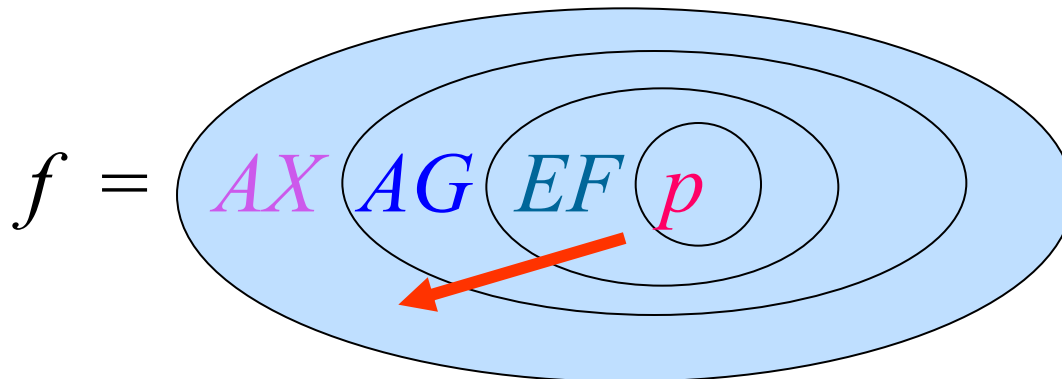
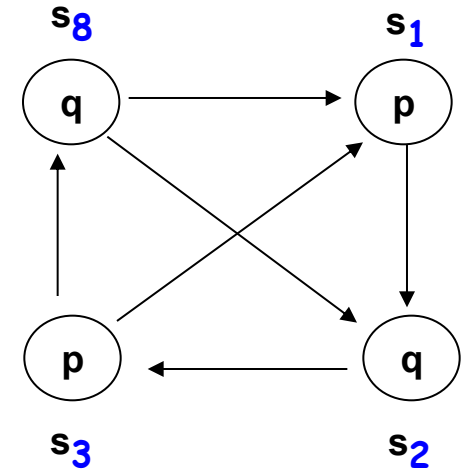


- $Label: S \rightarrow$ set of sub-formulae of f .
- Label each state s with the set $Label(s)$ of sub-formulae of f which are true in s .
- $\forall s \in S. Label(s) \supseteq L(s)$.
- Initially, $Label(s) \leftarrow L(s)$.

- $\{s_1, s_3\} \models p$
- $\{???\} \models AFEX p$

Structure of Formulas

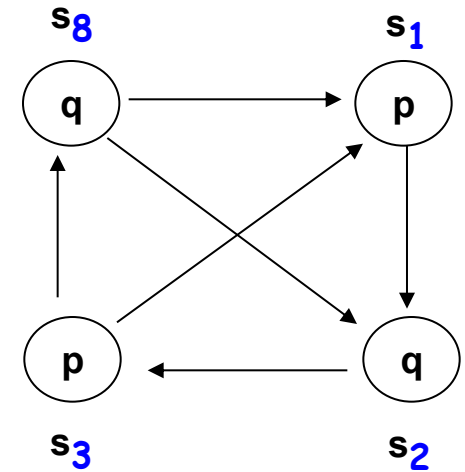
- Nested pairing
- Starting with the **smallest sub-formulas**
- Working outwards towards f .
- Label the states of M with the sub-formulas of f that are satisfied.



- $\{s_1, s_3\} \models p$
- $\{???\} \models AFEX p$

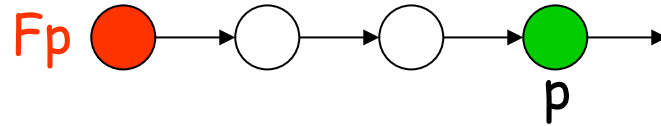
MC Algorithms: Labeling

- Compute the set of states that satisfy the atomic formulae
- Proceed to more complicated subformulas.
- Let g be a sub-formula of f and states satisfying all the immediate sub-formulas of g have been labeled.
- $S(g)$ = the set of all states labeled by the subformula g .

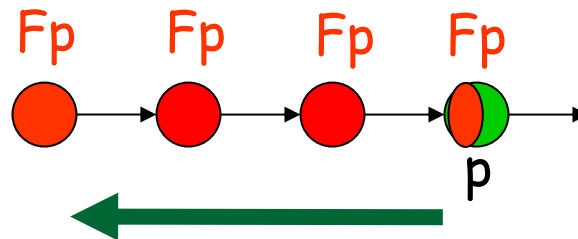
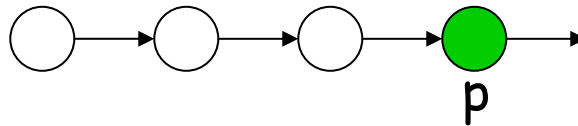


- $\{s_1, s_3\} \models p$
- $\{ ??? \} \models \text{AFEX } p$

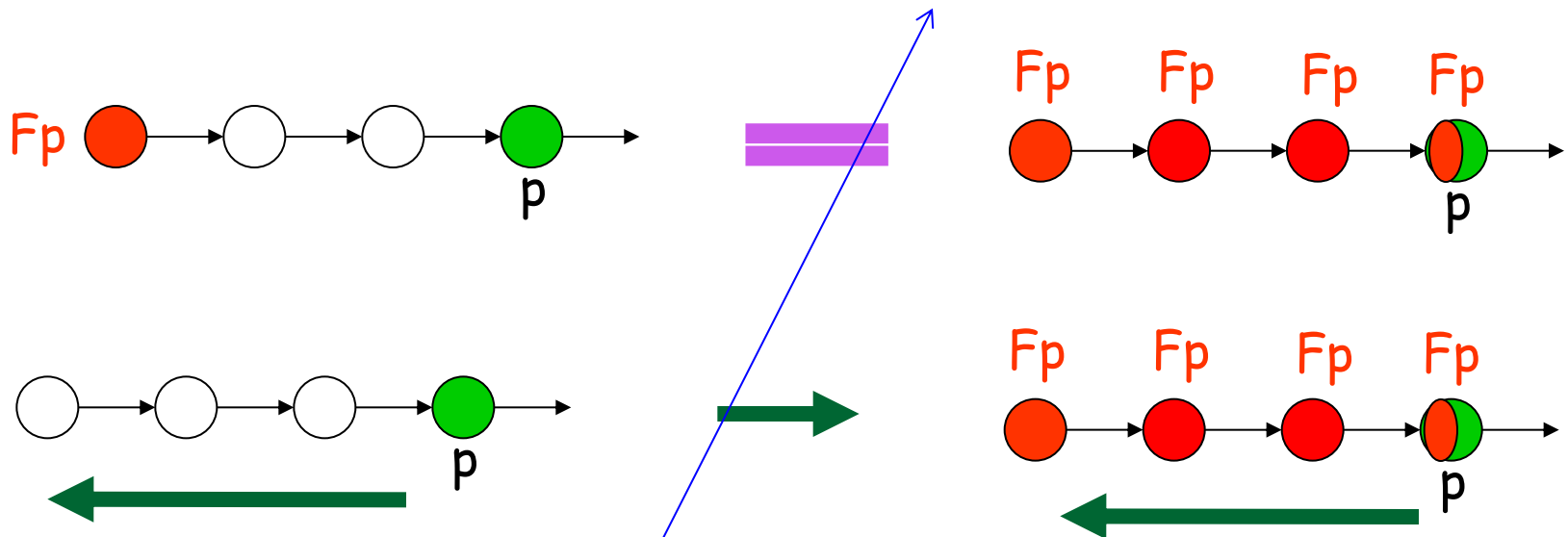
Semantics



- If we know where p is true, then find where Fp is true.



Labeling: p to $EF\ p$

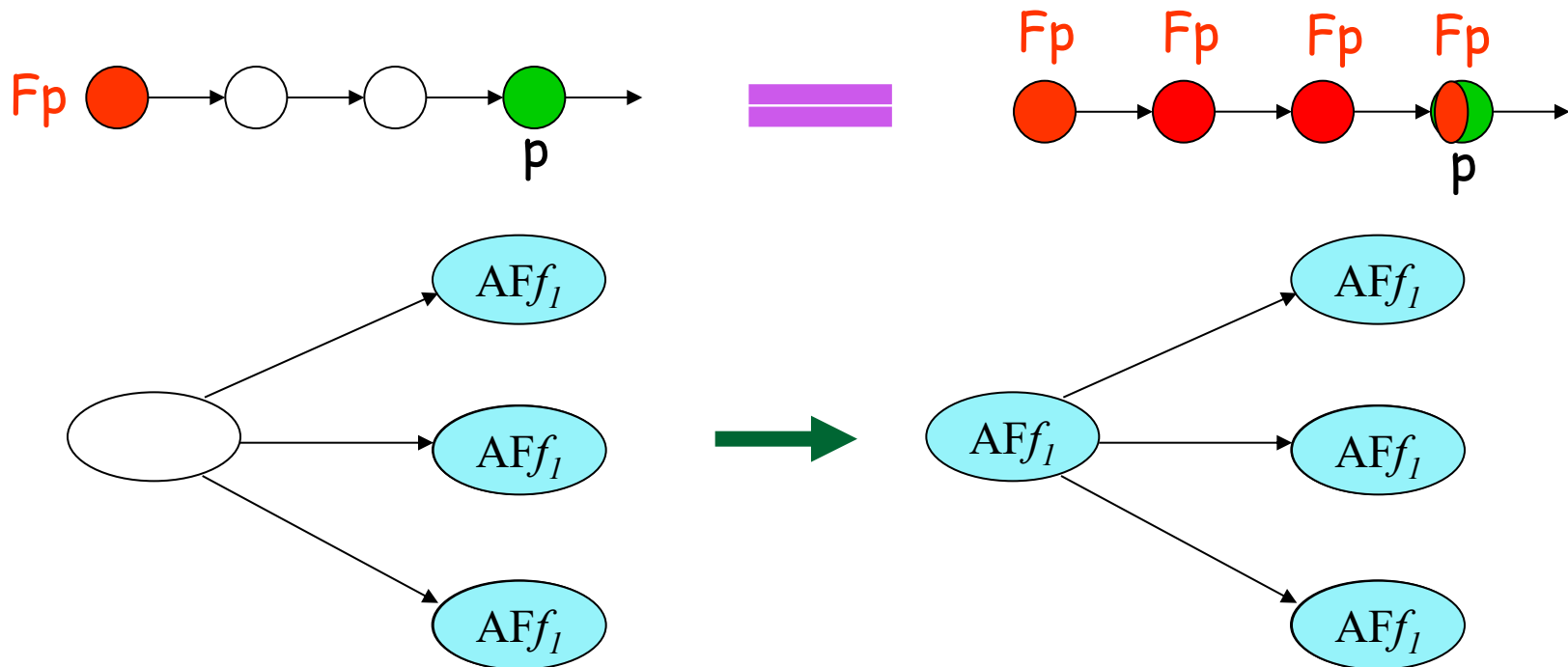


- To successively apply the state labeling algorithm to the sub-formulas
 - start with the shortest, most deeply nested
 - work **outwards on formula** and **backwards on M** .

MC Algorithms: Labeling

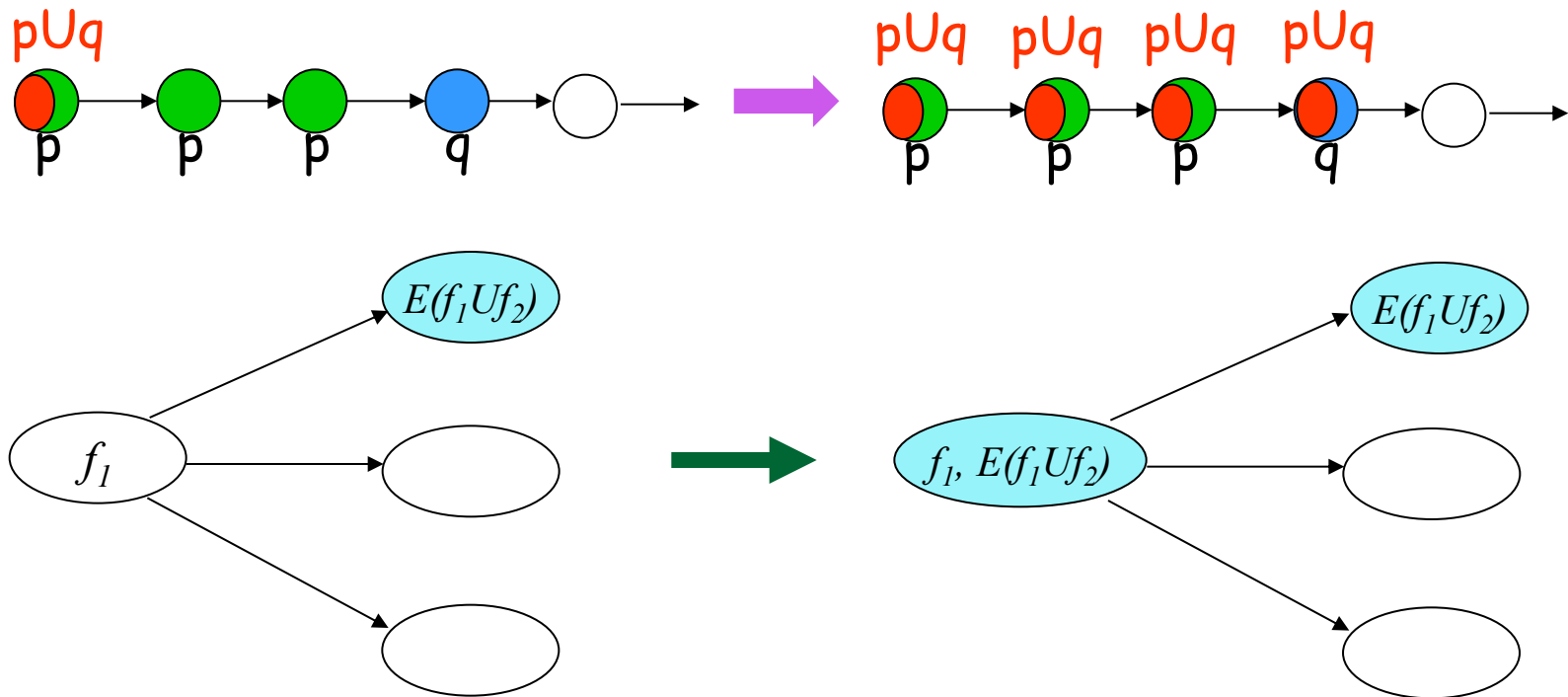
- Determine which states to label with g by a case analysis:
 - $g = \text{False}$
No states are labeled with False.
 - $g = p$
Label s with p if $p \in L(s)$.
 - $g = f_1 \wedge f_2$
Label s with $f_1 \wedge f_2$ if s is already labeled both with f_1 and with f_2 .
 - $g = \neg f_1$
Label s with $\neg f_1$ if s is not already labeled with f_1 .
 - $g = \text{EX } f_1$
label any state with $\text{EX}f_1$ if one of its successors is labeled with f_1 .

Labeling: $g = AF f_1$



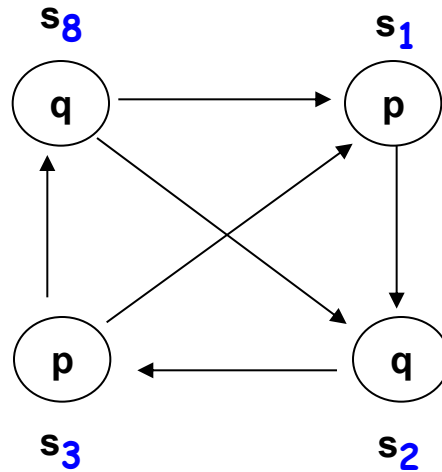
- If any state s is labeled with f_1 , label it with $AF f_1$.
- Repeat: label any state with $AF f_1$ if all successor states are labeled with $AF f_1$, until there is no change.

Labeling: $g = E[f_1 U f_2]$



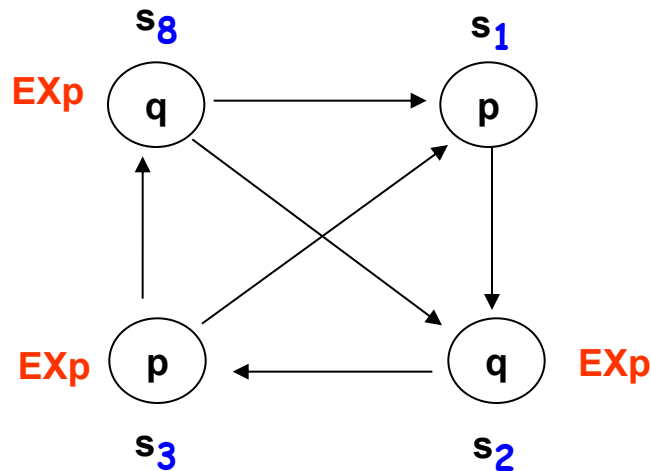
- If any state s is labeled with f_2 , label it with $E[f_1 U f_2]$.
- Repeat: label any state with $E[f_1 U f_2]$ if it is labeled with f_1 and at least one of its successors is labeled with $E[f_1 U f_2]$, until there is no change.

Example



- $Q = \text{AF } \textcolor{red}{EX} \textcolor{black}{p}$
- $\{ ??? \} \models p \rightarrow \{ \textcolor{blue}{s}_1, \textcolor{blue}{s}_3 \} \models p$

Example



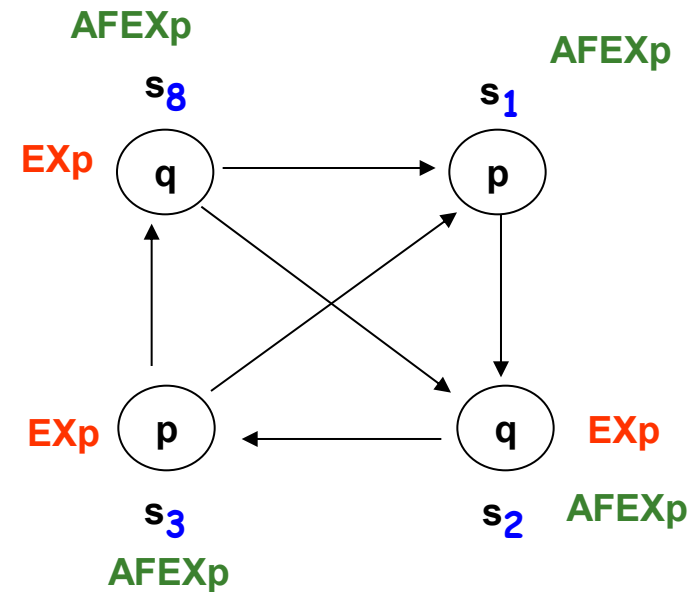
- $g = EX f_1$
- label any state with EXf_1 if one of its successors is labeled with f_1 .

- $Q = AF \text{ EX } p$
- $\{ ??? \} \models p \rightarrow \{ s_1, s_3 \} \models p$
- $\{ ??? \} \models EXp$
 - $\rightarrow \{ s_8, s_2, s_3 \} \models EXp$

Example

- If any state s is labeled with f_1 , label it with $AF f_1$.
- Repeat: label any state with $AF f_1$ if all successor states are labeled with $AF f_1$, until there is no change.

- $Q = AF EX p$
- $\{ ??? \} \models p \rightarrow \{ s_1, s_3 \} \models p$
- $\{ ??? \} \models EXp$
 - $\rightarrow \{ s_8, s_2, s_3 \} \models EXp$
- $\{ ??? \} \models AF EXp$
 - $\rightarrow \{ s_8, s_1, s_2, s_3 \} \models AF EXp$
- $\{ ??? \} \models AX EXp = \{ s_1, s_2 \}$



Basic Idea: Labeling

- The algorithm goes through a series of stages.
 - During the i -th stage, sub-formulae with $i-1$ nested CTL operators are processed.
 - When a sub-formula is processed, it is added to the labeling of each state in which it is true.
 - Once the algorithm terminates, we will have that:

$$M, s \models f \text{ iff } f \in \text{Label}(s).$$

MC Algorithms: Labeling

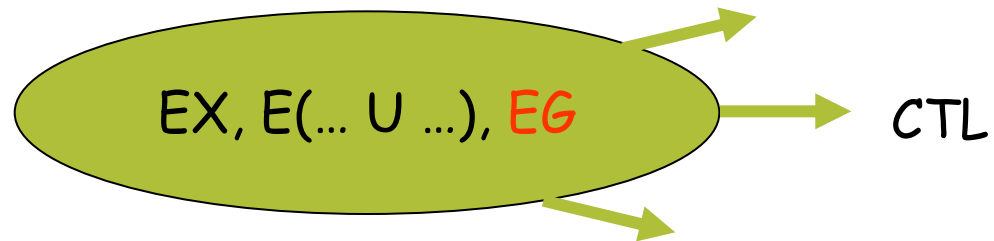
```
function MC ( f )    /* determine the set of states satisfying f */
begin
  case
    f is True:      return S;
    f is False:     return  $\emptyset$ ;
    f is atomic:    return  $\{s \in S \mid f \in L(s)\}$ ;
    f is  $\neg f_1$ :    return  $S - MC(f_1)$ ;
    f is  $f_1 \wedge f_2$ : return  $MC(f_1) \cap MC(f_2)$ ;
    f is  $f_1 \vee f_2$ : return  $MC(f_1) \cup MC(f_2)$ ;
    f is  $f_1 \rightarrow f_2$ : return  $MC(\neg f_1 \vee f_2)$ ;
    f is  $AXf_1$ :     return  $MC(\neg EX \neg f_1)$ ;
    f is  $EXf_1$ :     return  $MC_{EX}(f_1)$ ;
    f is  $A(f_1 U f_2)$ : return  $MC(\neg (E[\neg f_2 U (\neg f_1 \wedge \neg f_2)] \vee EG \neg f_2))$ ;
    f is  $E(f_1 U f_2)$ : return  $MC_{EU}(f_1, f_2)$ ;
    f is  $EFf_1$ :     return  $MC(E(\text{True} U f_1))$ ;
    f is  $EGf_1$ :     return  $MC(\neg AF \neg f_1)$ ;
    f is  $AFf_1$ :     return  $MC_{AF}(f_1)$ ;
    f is  $AGf_1$ :     return  $MC(\neg EF \neg f_1)$ ;
  end case
end function
```

MC Algorithms: Labeling

- Having performed the labeling for all the sub-formulas of f (including f itself), output the states which are labeled f .
- The complexity is $O(|f| * |S| * (|S| + |R|))$, where
 - $|f|$ is the number of connectives in the formula
 - $|S|$ is the number of states
 - $|R|$ is the number of transitions
- The algorithm is
 - *linear* in the size of the formula
 - *quadratic* in the size of the model.

Another CTL Generator

AF, EU, EX, $\wedge$, $\neg$ and False



$$EFp = E(\text{true} \cup p)$$

$$AXp = \neg EX \neg p$$

$$AGp = \neg EF \neg p$$

$$A[p \cup q] = \neg \{E[\neg q \cup (\neg p \wedge \neg q)] \vee EG \neg q\}$$

MC Algorithms: Labeling-2

$f =$ **EG**, EU, EX, $\wedge$, $\neg$ and False

MC Algorithms: Labeling-2

- It is sufficient to handle six cases: depending on whether
 - g is atomic or
 - one of the following forms:
 $\neg f_1$, $f_1 \vee f_2$, EXf_1 , $E(f_1 \cup f_2)$, or EGf_1 .
- For formulae of the form $\neg f_1$,
 - label those states that are not labeled by f_1 .
- For $f_1 \vee f_2$, label any state that is labeled either by f_1 or f_2 .
- For EXf_1 , label every state that has some successor labeled by f_1 .

$$g = E[f_1 \cup f_2]$$

- For EU, search the model by backwards breadth-first searching.
 - This ensures that we **do not pass over any node twice**.
- Basic Idea:
 - find all states that are labeled with f_2 .
 - work **backwards** using the converse of the transition relation R and
 - find all states that can be reached by a path in which each state is labeled with f_1 .
 - All such states is labeled with g .

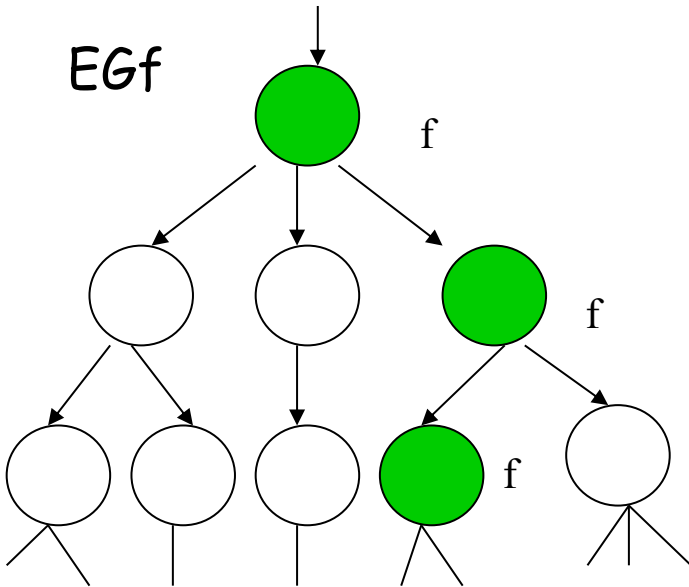
$$g = E[f_1 \cup f_2]$$

```
procedure CheckEU (  $f_1, f_2$  )  
   $T := \{s \mid f_2 \in \text{label}(s)\};$   
  for all  $s \in T$  do  $\text{label}(s) := \text{label}(s) \cup \{E[f_1 \cup f_2]\};$   
  while  $T \neq \emptyset$  do  
    choose  $s \in T$ ;  
     $T := T \setminus \{s\};$   
    for all  $t$  such that  $R(t, s)$  do  
      if  $E[f_1 \cup f_2] \notin \text{label}(t)$  and  $f_1 \in \text{label}(t)$  then  
         $\text{label}(t) := \text{label}(t) \cup \{E[f_1 \cup f_2]\};$   
         $T := T \cup \{t\};$   
      end if;  
    end for all;  
  end while;  
end procedure
```

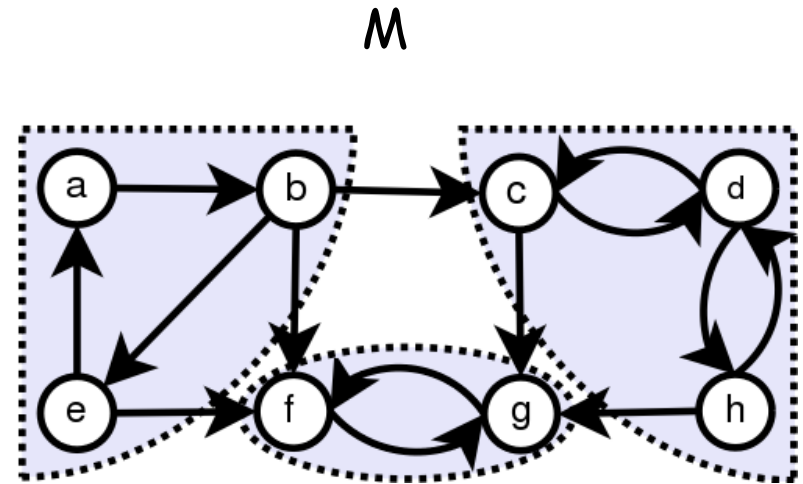

$$g = E[f_1 \cup f_2]$$

- The procedure *CheckEU* adds $E[f_1 \cup f_2]$ to $label(s)$ for every s that satisfies $E[f_1 \cup f_2]$, assuming that f_1 and f_2 have been processed correctly.
- For every state s , $f_1 \in label(s)$ iff $s \models f_1$ and $f_2 \in label(s)$ iff $s \models f_2$.
- The procedure requires time $O(|S| + |R|)$.

Characterization: $g = EG f$

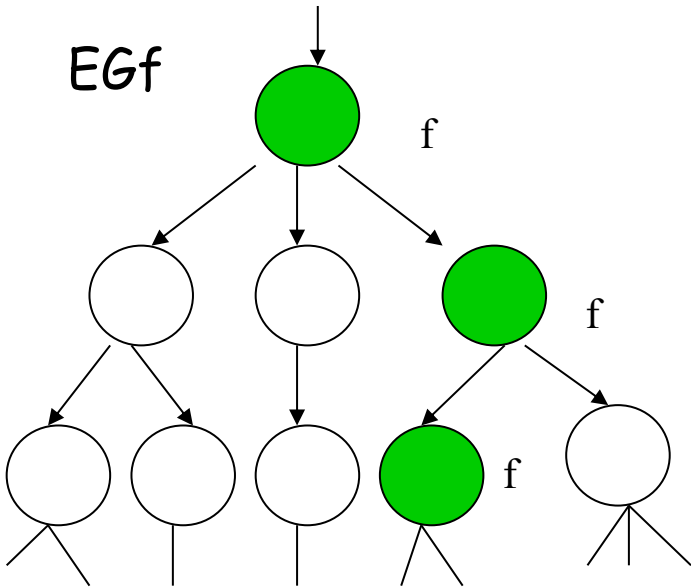


B: Semantic Tree

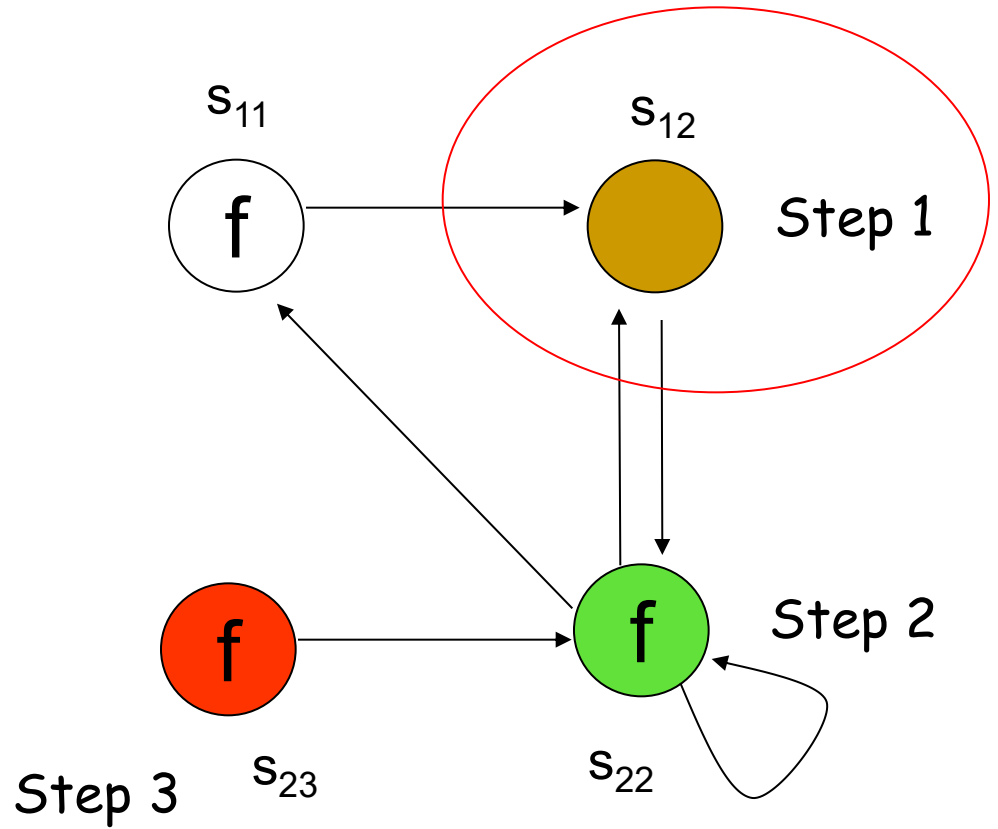


If a node is qualified for the root of B, what kind of structure should be in M?

Step 1: Model Reduction



B: Semantic Tree



Restriction on S

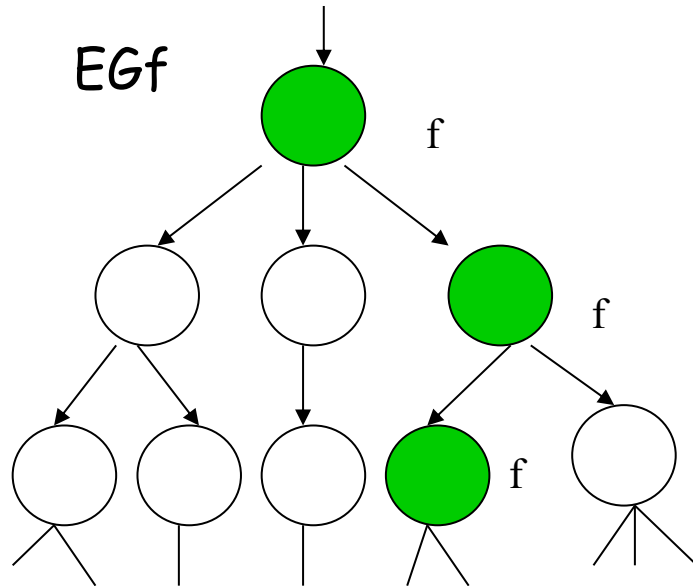
- Let M' be obtained from deleting from S all of those states at which f does not hold and restricting R and L accordingly.
- Thus $M' = (S', R', L')$ where

$$S' = \{ s \in S \mid M, s \models f \}$$

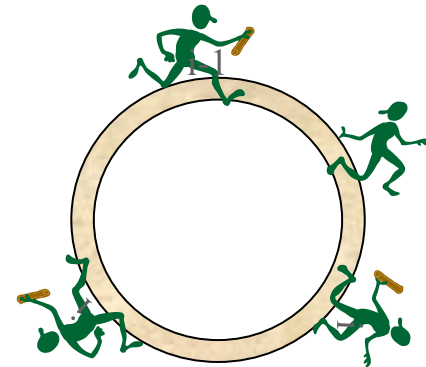
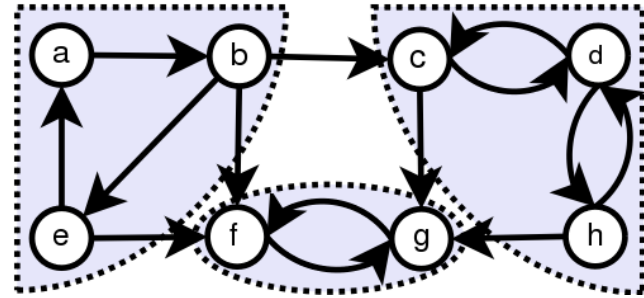
$$R' = R \mid_{S' \times S'}$$

$$L' = L \mid_{S'}$$

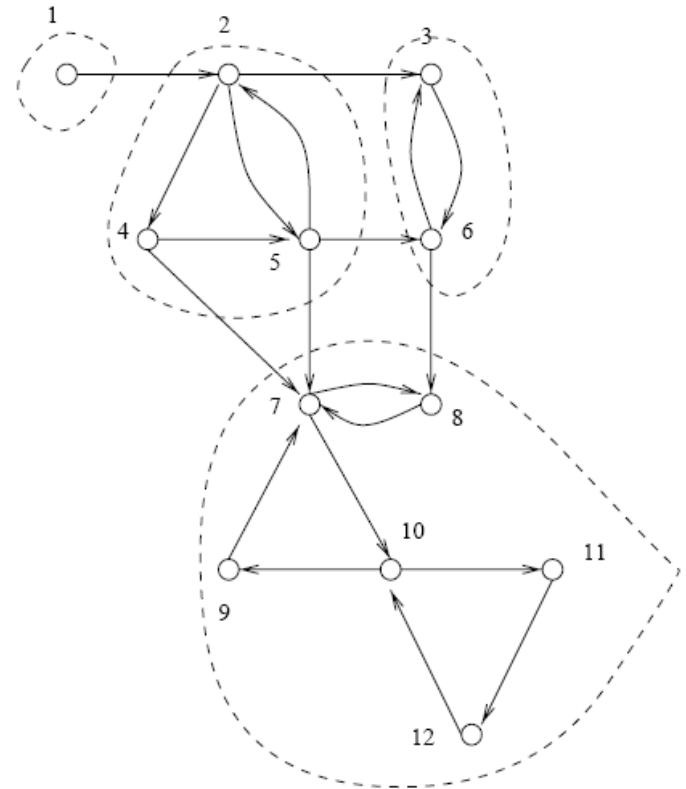
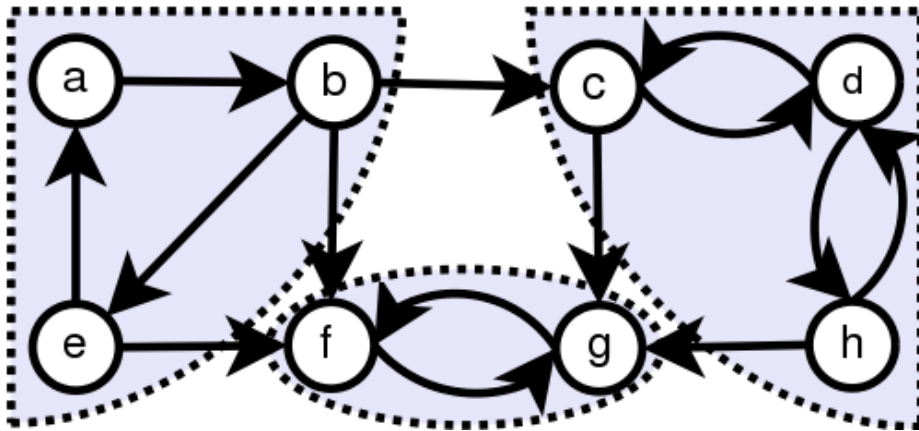
Step 2: Infinite Paths = Loops !



B: Semantic Tree



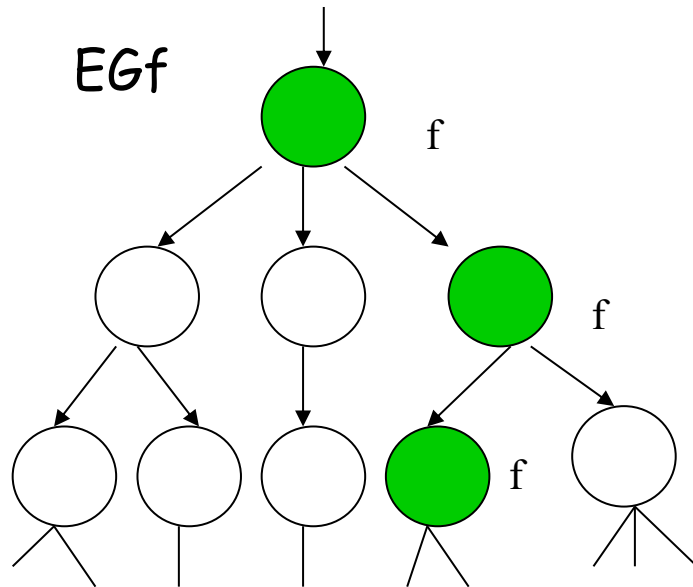
What is a Loop?



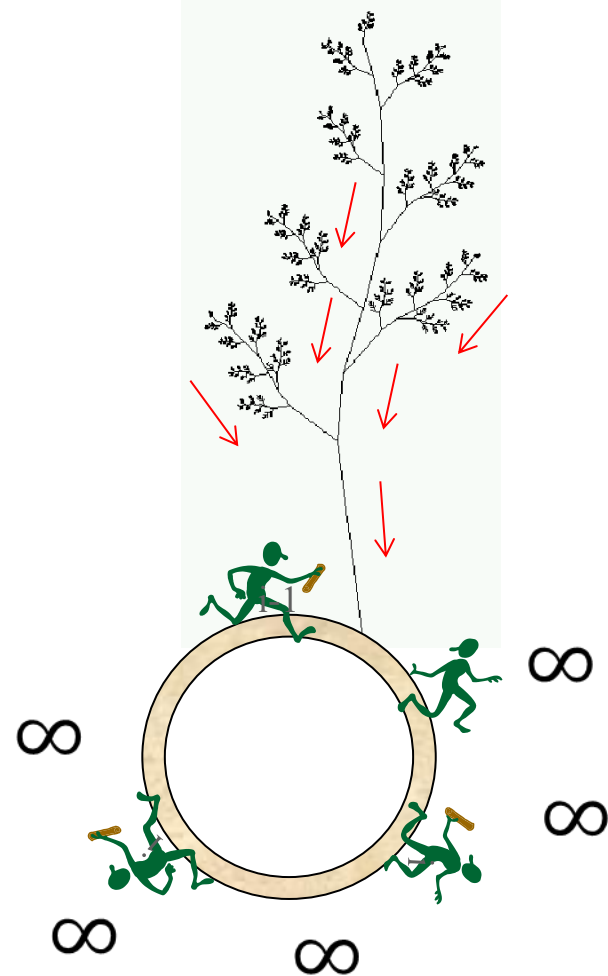
Loops = SCCs

- Strongly Connected Components = SCCs
- A SCC C is a maximal sub-graph such that
 - every node in C is reachable from every other node in C along a directed path entirely contained within C .

Step 3: Don't Forget Us !

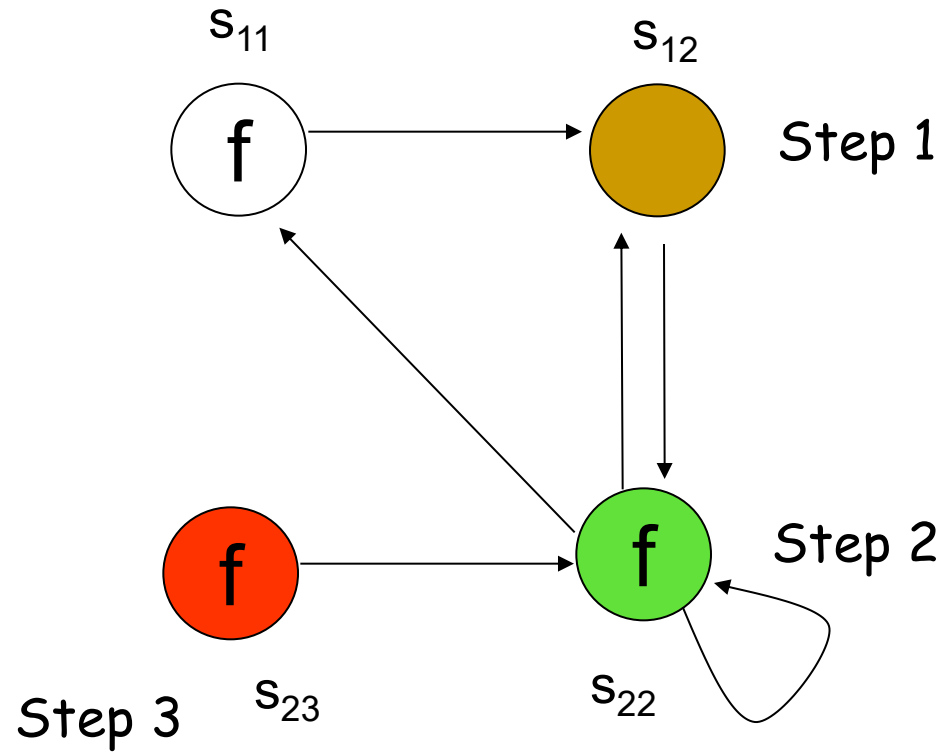
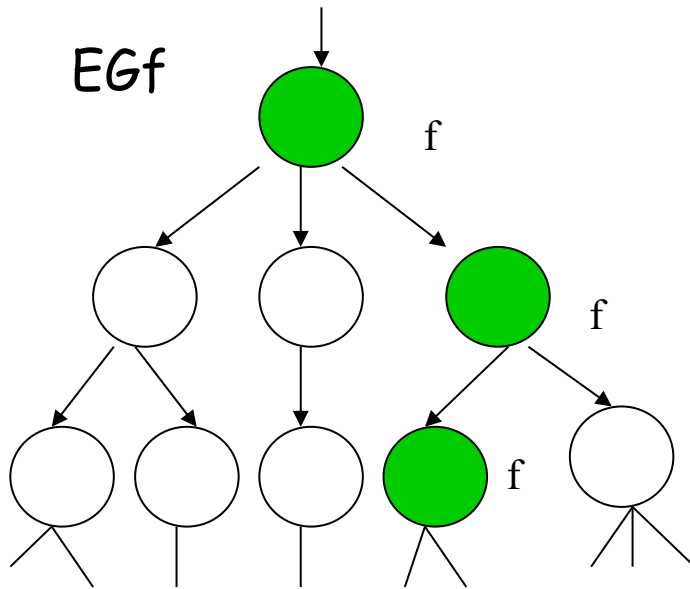


B: Semantic Tree

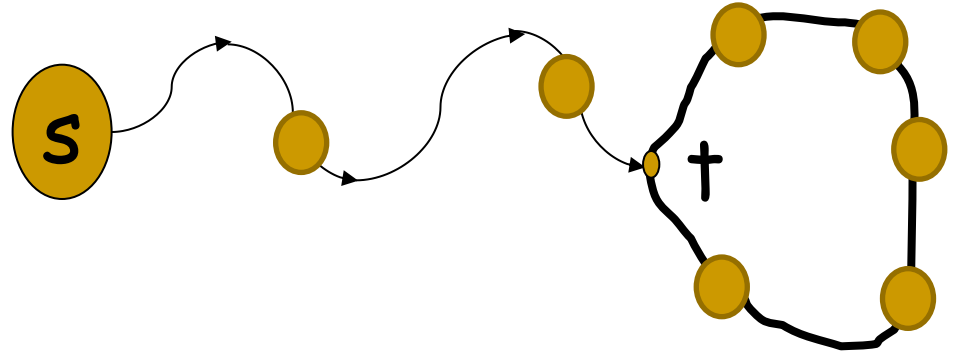
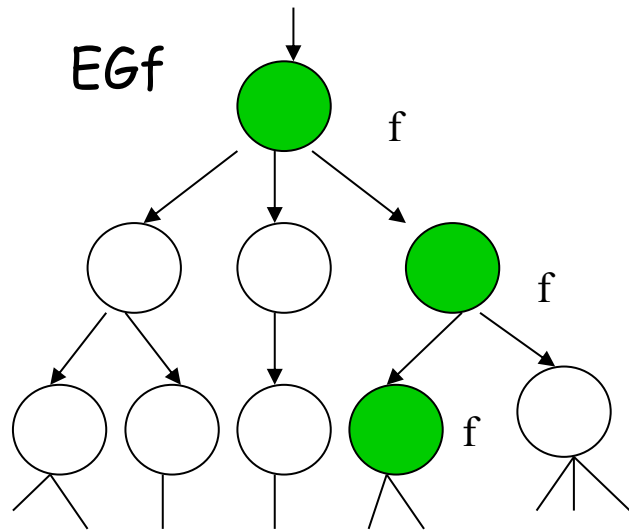


Such structures with f are needed !

Steps : $g = EGf$



Theorem for EG f



■ Theorem:

$M, s \models EG f$ iff the following two conditions are satisfied:

- $s \in S'$.
- a path $\in M'$ that leads from s to some node t in a non-trivial strongly connected component C of the graph (S', R') .

Proof: Part 1

- IF $M, s \models EG f$
- THEN
 - $s \in S'$.
 - a path $\in M'$ that leads from s to some node t in a non-trivial strongly connected component C of the graph (S', R') .

Proof.

1) Assume: $M, s \models EG f \Rightarrow s \in S'$ (as $M, s \models f$)

Find Structure C

2) Let π be an **infinite** path starting at s such that f holds at each state on π

$\Rightarrow$ since **M is finite**, it must be possible to write π as

$\pi = \pi_0 \pi_1$ where

π_0 is a finite initial segment and

π_1 is an infinite suffix of π with the property that **each state on π_1 occurs infinitely often**.

$\Rightarrow$ Let C be the set of states in π_1 .

$\Rightarrow C$ is contained in S' (as $\forall s_i, M, s_i \models f$)

Characterize C as an SCC

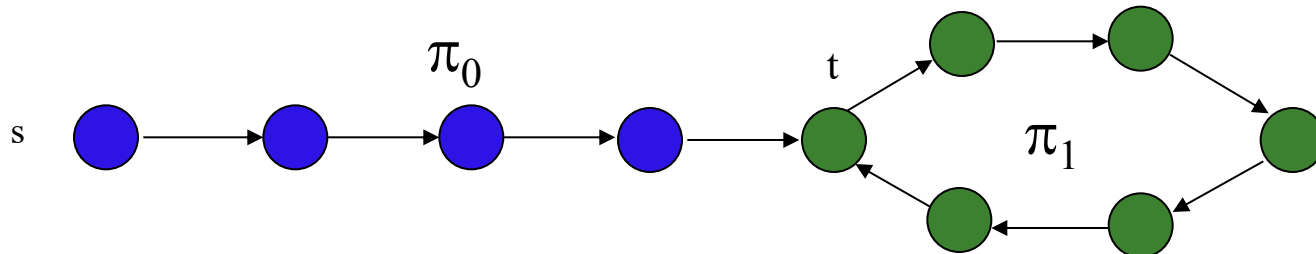
- Show that: $\exists$ a path within C between any pair of states in C .
- Let s_1 and s_2 be states in C .
 - $\Rightarrow$ Pick s_1 on π_1 .
 - $\Rightarrow$ there is s_2 further along π_1 (on the same time line)
 - $\Rightarrow$ the segment from s_1 to s_2 lies entirely within C
 - $\Rightarrow$ this segment is a finite path from s_1 to s_2 in C
 - $\Rightarrow$ either C is a strongly connected component or it is contained within one
 - $\Rightarrow$ in either case, both conditions (1) and (2) are satisfied.

Proof Part 2

- IF
 - $s \in S'$.
 - a path $\in M'$ that leads from s to some node t in a non-trivial strongly connected component C of the graph (S', R') .
- THEN
 - $M, s \models EG f$

Proof Part 2

- Assume that conditions (1) and (2) are satisfied.
- Let π_0 be the path from s to t . (π_0 may be empty)
 - $\Rightarrow$ Let π_1 be a finite path of length ≥ 1 that leads from t back to t .
 - $\Rightarrow$ The existence of π_1 is guaranteed because t is a state in a non-trivial strongly connected component.
 - $\Rightarrow$ All the states on the infinite path $\pi = \pi_0 \pi_1^\omega$ satisfy f_1 .
 - $\Rightarrow$ Since π is also a possible path starting at s in M .
 - $\Rightarrow$ $M, s \models EG f$.

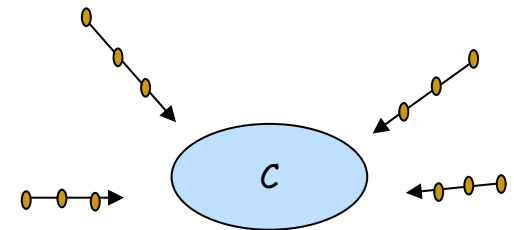
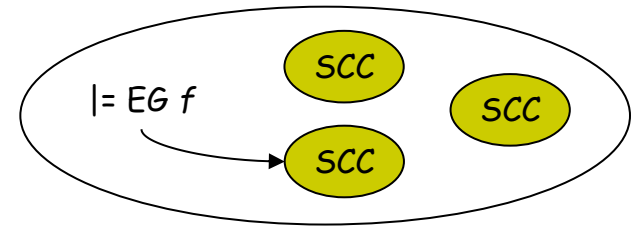


Algorithm EGf

- Step 1: Restrict the graph to states satisfying f , i.e.,
 - delete all other states and their transitions
- Step 2: Find the maximal SCCs
 - These are maximal regions of the state space in which every state is linked (= has a finite path to) every other one in that region.
- Step 3: Use backwards breadth-first searching on the restricted graph to find any state that can reach an SCC.

$$\models EG f$$

- Step 1: Construct $M' = (S', R', L')$
- Step 2:
 - Compute the set of $\{C_1, C_2, \dots, C_k\}$ of M' .
 - Partition the graph (S', R') into SCCs using Tarjan's Algorithm
 - Time complexity: $O(|S'| + |R'|)$
 - $C = C_1 \cup C_2 \cup \dots \cup C_k$
- Step 3: Compute the set of states satisfying $E(f \cup C)$
 - Find those states belonging to non-trivial components C_i .
 - Work backward using the converse of R'
 - find all of those states that can be reached by a path in which each state is labeled with f and mark them with EGf .
- Time complexity: $O(|S| + |R|)$



$$g = EG f$$

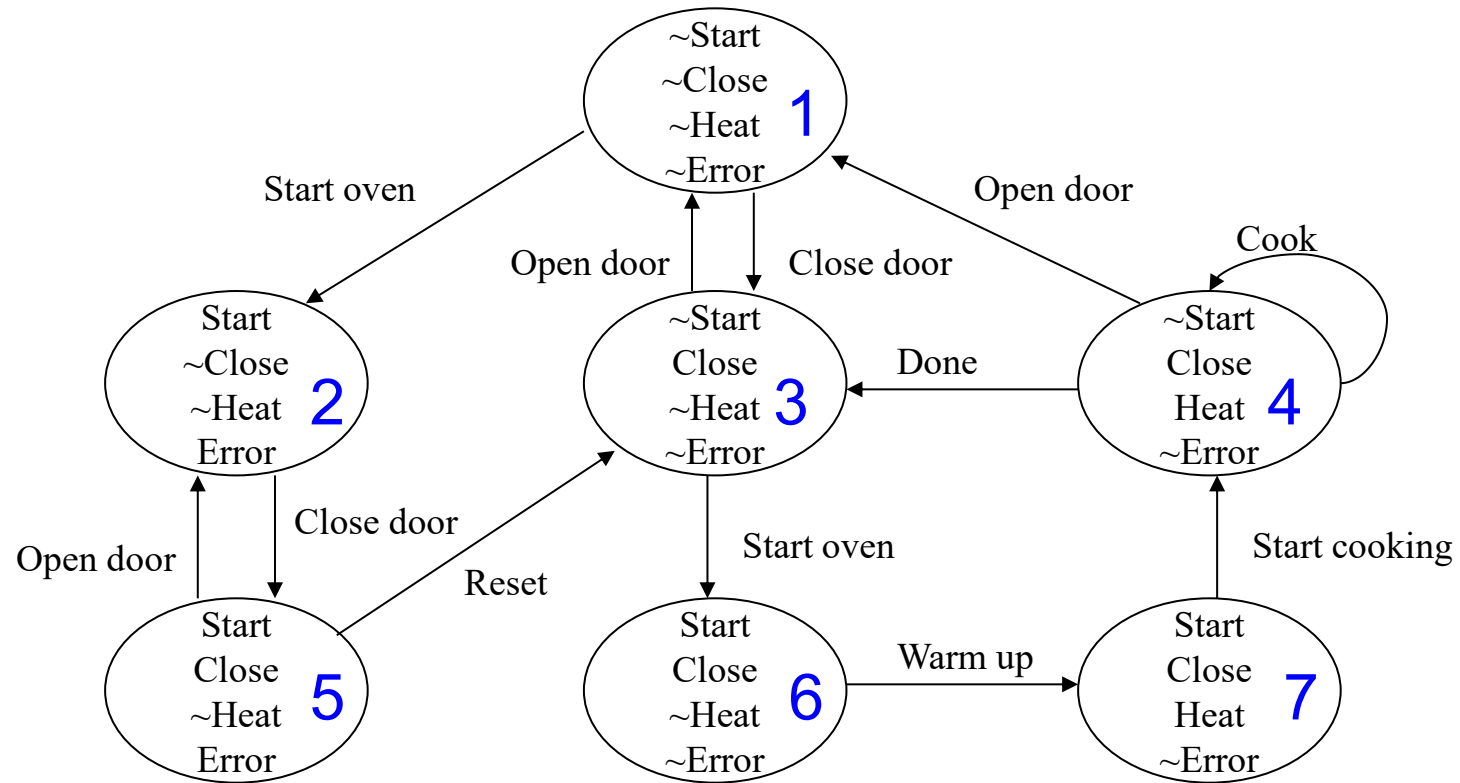
```
procedure CheckEG(f)
   $S' := \{s \mid f \in \text{label}(s)\};$ 
   $SCC := \{C \mid C \text{ is a nontrivial SCC of } S'\};$ 
   $T := \bigcup_{C \in SCC} \{s \mid s \in C\};$ 
  for all  $s \in T$  do  $\text{label}(s) := \text{label}(s) \cup \{\mathbf{EG} f\};$ 
  while  $T \neq \emptyset$  do
    choose  $s \in T$ ;
     $T := T \setminus \{s\};$ 
    for all  $t$  such that  $t \in S'$  and  $R(t,s)$  do
      if  $\mathbf{EG} f \notin \text{label}(t)$  then
         $\text{label}(t) := \text{label}(t) \cup \{\mathbf{EG} f\};$ 
         $T := T \cup \{t\};$ 
      end if;
    end for all;
  end while;
end procedure
```

Model Checking by Labeling

- To handle an arbitrarily CTL formula f , we apply the state-labeling algorithm to the sub-formulae of f ,
 - starting with the shortest, most deeply nested, and work outward to include all of f .
- Each pass takes time: $O(|S| + |R|)$.
- Since f has at most $|f|$ different sub-formulae, the entire algorithm requires time: $O(|f| * (|S| + |R|))$.
- *The algorithm is linear both in the size of the model and in the size of the formula.*

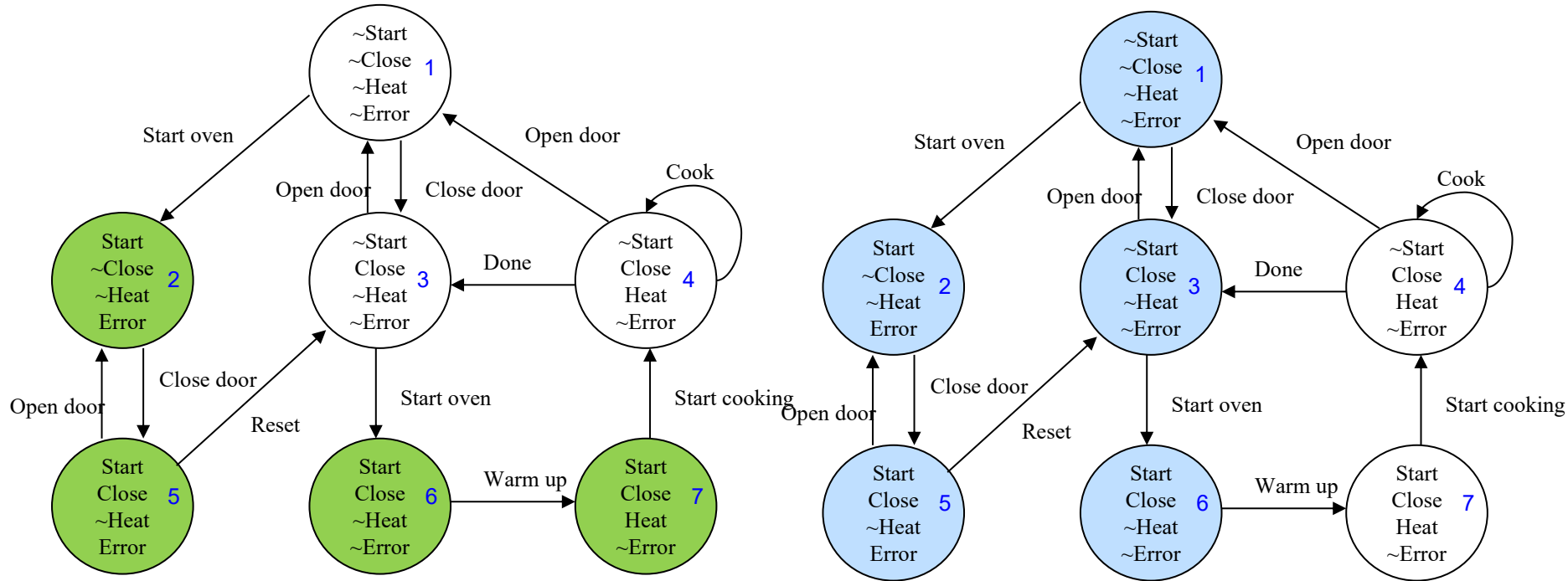
The complexity is $O(|f| * |S| * (|S| + |R|))$,

Microwave Oven Controller



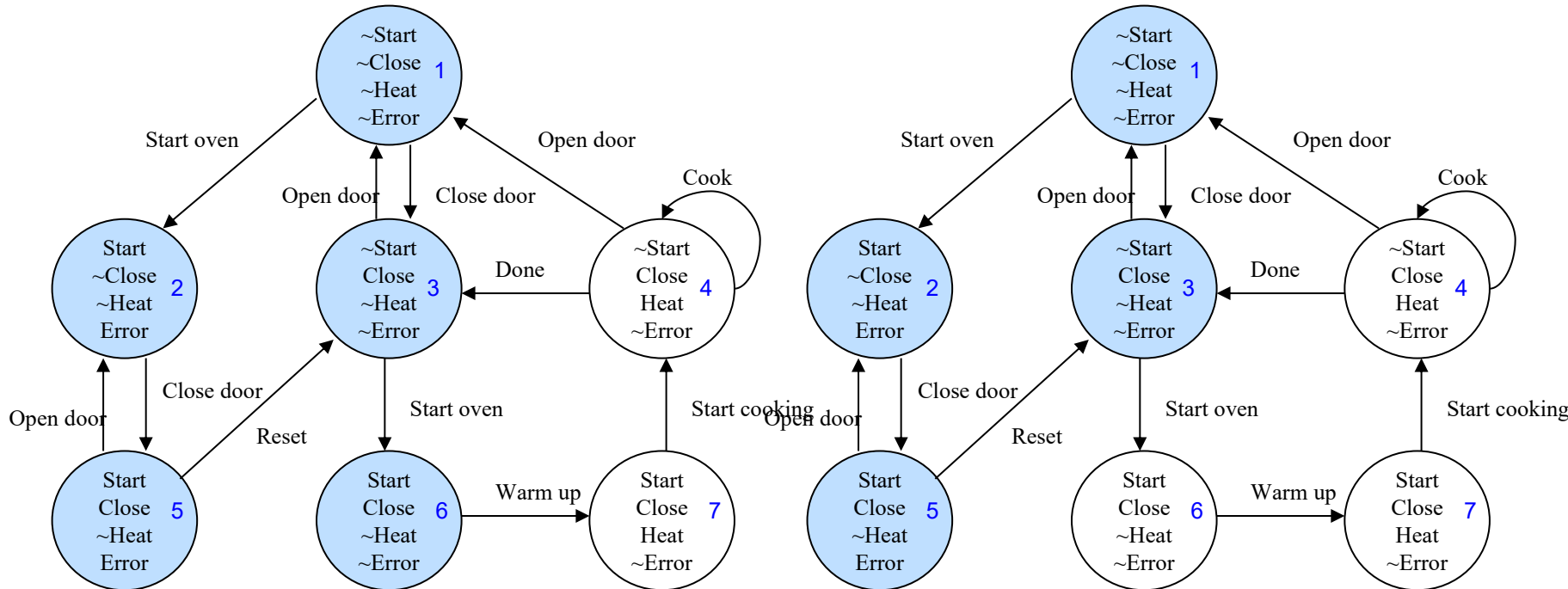
[Clark99]

AG (Start $\rightarrow$ AF Heat)



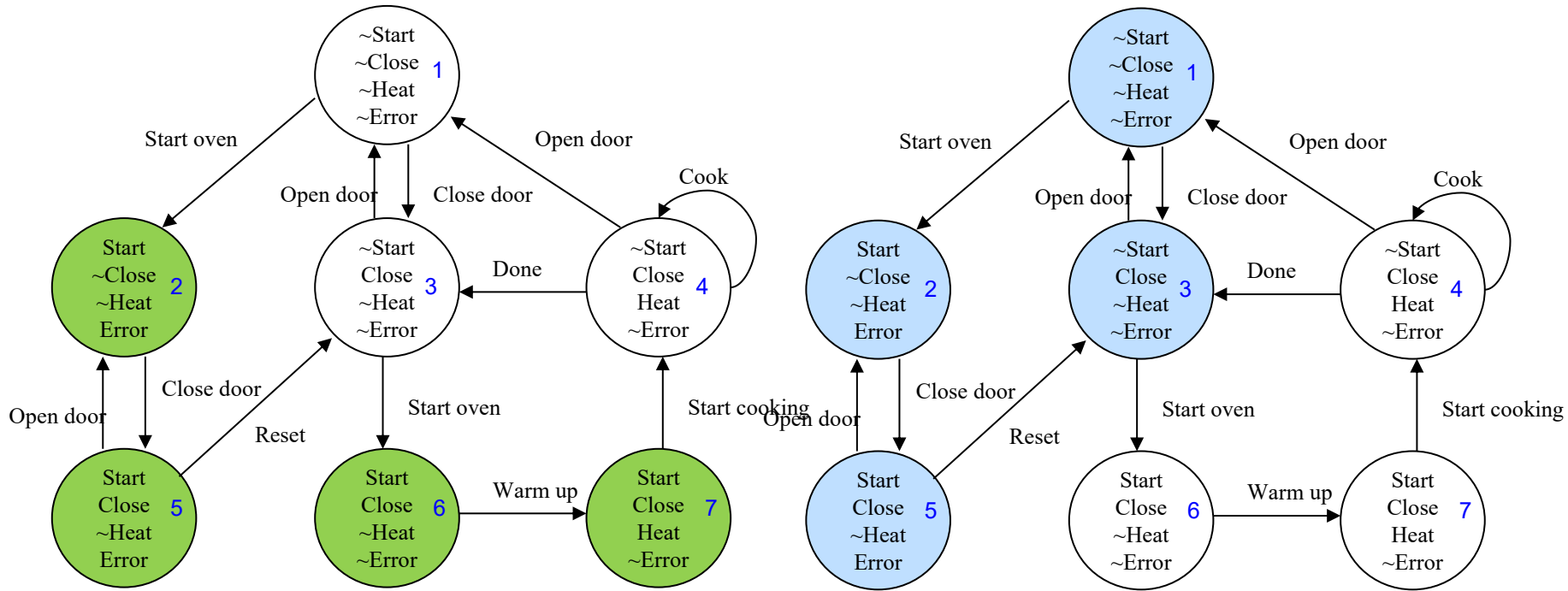
- $AG (Start \rightarrow AF Heat) = \neg EF (Start \wedge EG \neg Heat)$
- $S(g)$ = the set of all states labeled by the subformula g .
- $S(Start) = \{2, 5, 6, 7\}$
- $S(\neg Heat) = \{1, 2, 3, 5, 6\}$

AG (Start $\rightarrow$ AF Heat)



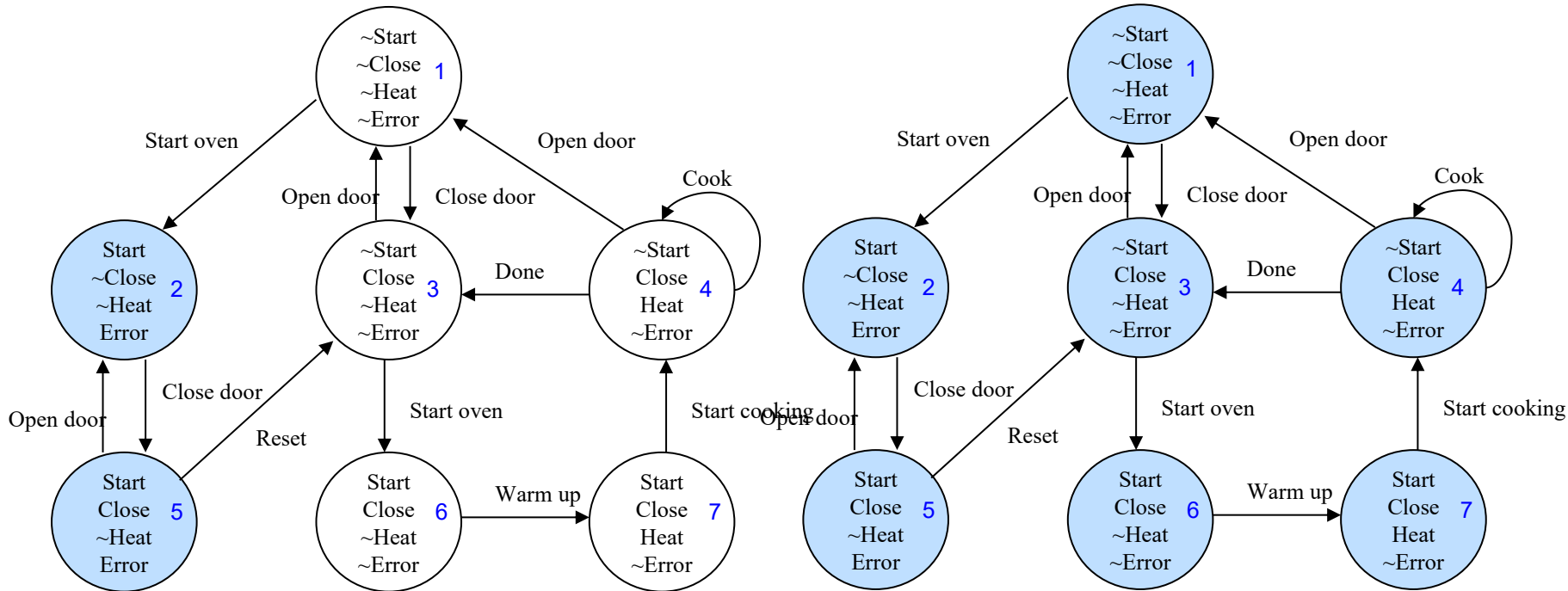
- Compute $S(EG \neg Heat)$
- Find the set of SCCs in $S' = S(\neg Heat)$: $SCC = \{\{1, 2, 3, 5\}\}$
- T = the set of all states labeled by $EG \neg Heat$. Initially, $T = \{1, 2, 3, 5\}$
- No other state in S' can reach a state in T along a path in S' .
- The computation terminates with $S(EG \neg Heat) = \{1, 2, 3, 5\}$

AG (Start $\rightarrow$ AF Heat)



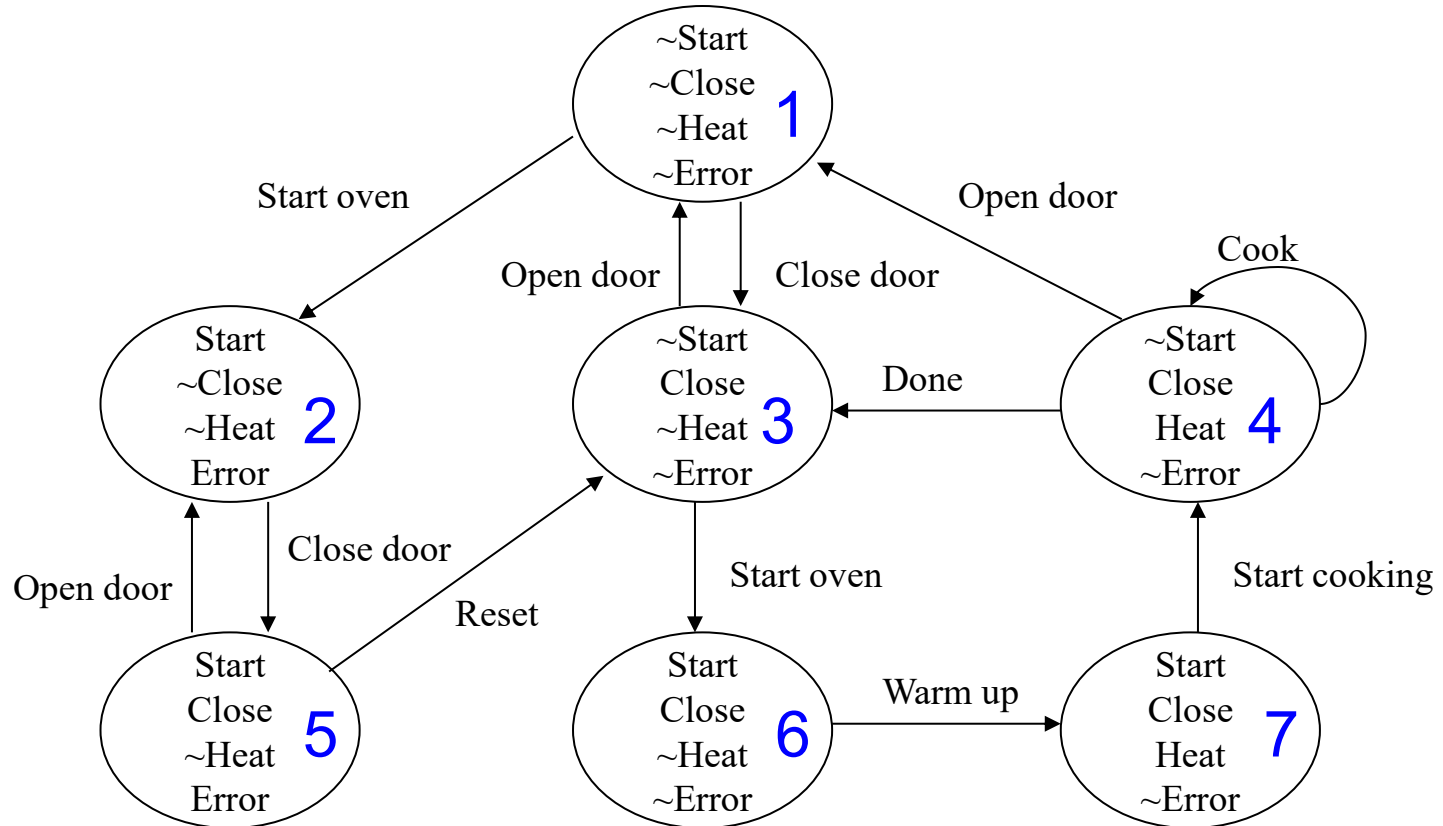
- $S(\text{Start}) = \{2, 5, 6, 7\}$
- $S(\text{EG} \neg \text{Heat}) = \{1, 2, 3, 5\}$
- $S(\text{Start} \wedge \text{EG} \neg \text{Heat}) = \{2, 5\}$.

AG (Start $\rightarrow$ AF Heat)



- $S(\text{Start} \wedge \text{EG} \neg \text{Heat}) = \{2, 5\}$.
- Compute: $S(\text{EF}(\text{Start} \wedge \text{EG} \neg \text{Heat}))$. Set $T = S(\text{Start} \wedge \text{EG} \neg \text{Heat})$
- Use the converse of the transition relation to label all the states in which the formula holds.
- $S(\text{EF}(\text{Start} \wedge \text{EG} \neg \text{Heat})) = \{1, 2, 3, 4, 5, 6, 7\}$

AG (Start $\rightarrow$ AF Heat)



- $S(\neg EF(\text{Start} \wedge EG \neg \text{Heat})) = \emptyset$
- The initial state 1 is not contained in this set, the system does not satisfy the given specification.

References

- K. L. McMillan, *Symbolic Model Checking*, Kluwer Academic Publishers, 1993.
- E. M. Clarke, O. Grumberg, and D. E. Long, "Model checking and abstraction", *Proc. 19th ACM Symp. on Principles of Programming Languages*, January 1992.
- E. M. Clark, O. Grumberg and D. A. Peled, "Model Checking", MIT Press, 1999.
- M. Huth, M. Ryan, "Logic in Computer Science: Modeling and Reasoning about Systems", Cambridge University Press, 2000.
- C. Seger, "An introduction to formal hardware verification", Technical Report, UBC.